

Syslab在数学建模的基础应用

全国大学生数学建模竞赛

徐德军

苏州同元软控信息技术有限公司

2024年8月15日



传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，
版权所有，侵权必究

CONTENTS

目录

Part 1

科学计算环境 MWORKS.Syslab

Part 2

软件安装与界面介绍

Part 3

脚本构建

Part 4

脚本调试

Part 5

结果后处理

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司
版权所有，侵权必究

Part 1

科学计算环境 MWORKS.Syslab

1. MWORKS产品架构
2. 科学计算环境 MWORKS.Syslab

未经许可，不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

1.1 MWORKS产品架构

信息物理系统建模仿真通用平台
(Syslab+Sysplorer)

各装备行业数字化工程支撑平台
(Sysbuilder+Sysplorer+Syslink)

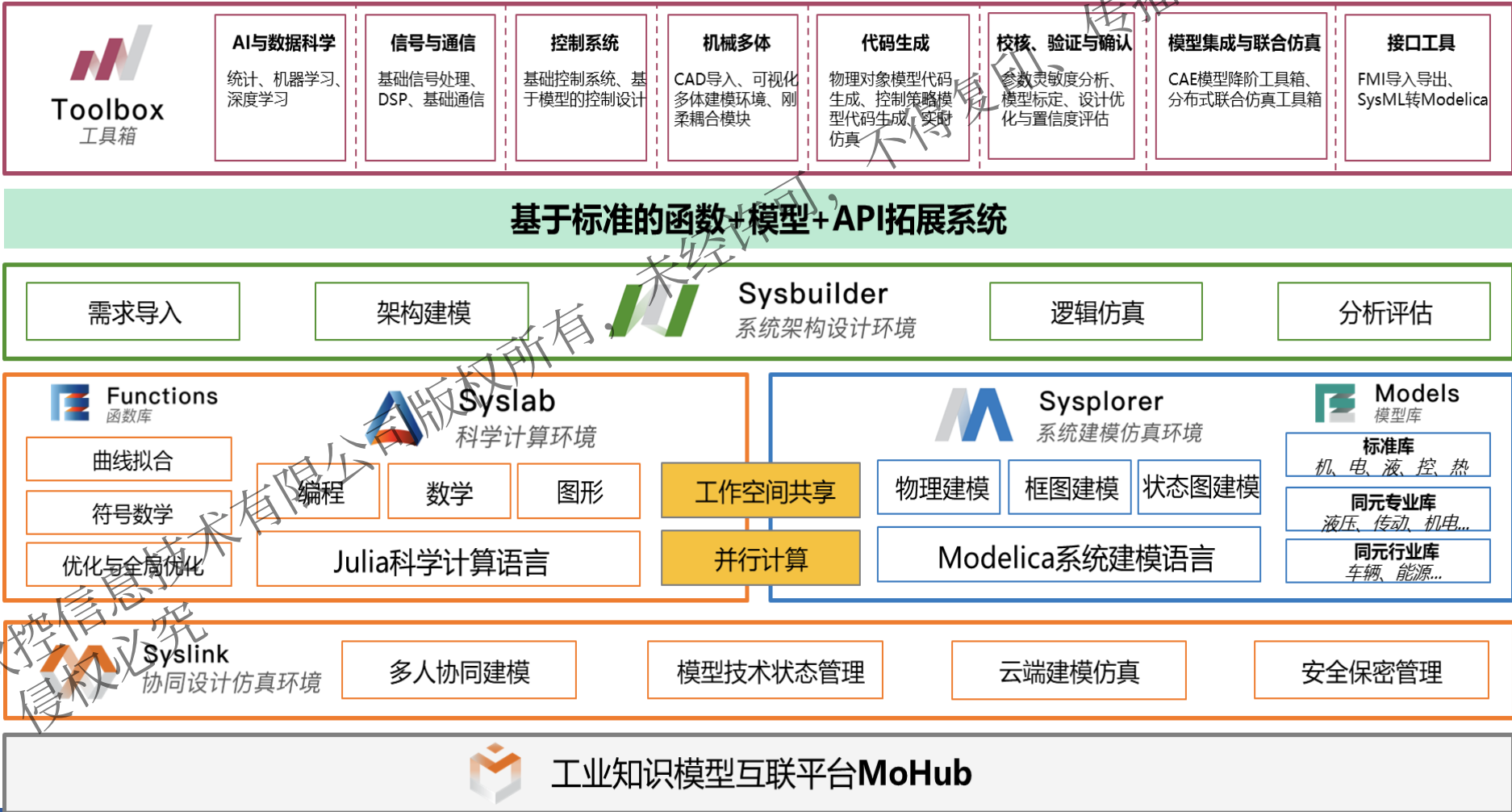
开放、标准、先进的计算仿真云平台
(MoHub)

N个领域工具

1套开放架构

4大系统软件

1个云环境



传播或以其他方式使用

工作区



通用编程与算法开发环境

基于高级通用动态编程语言支撑高性能算法开发

数据分析与可视化

对数据进行探查、分析与可视化

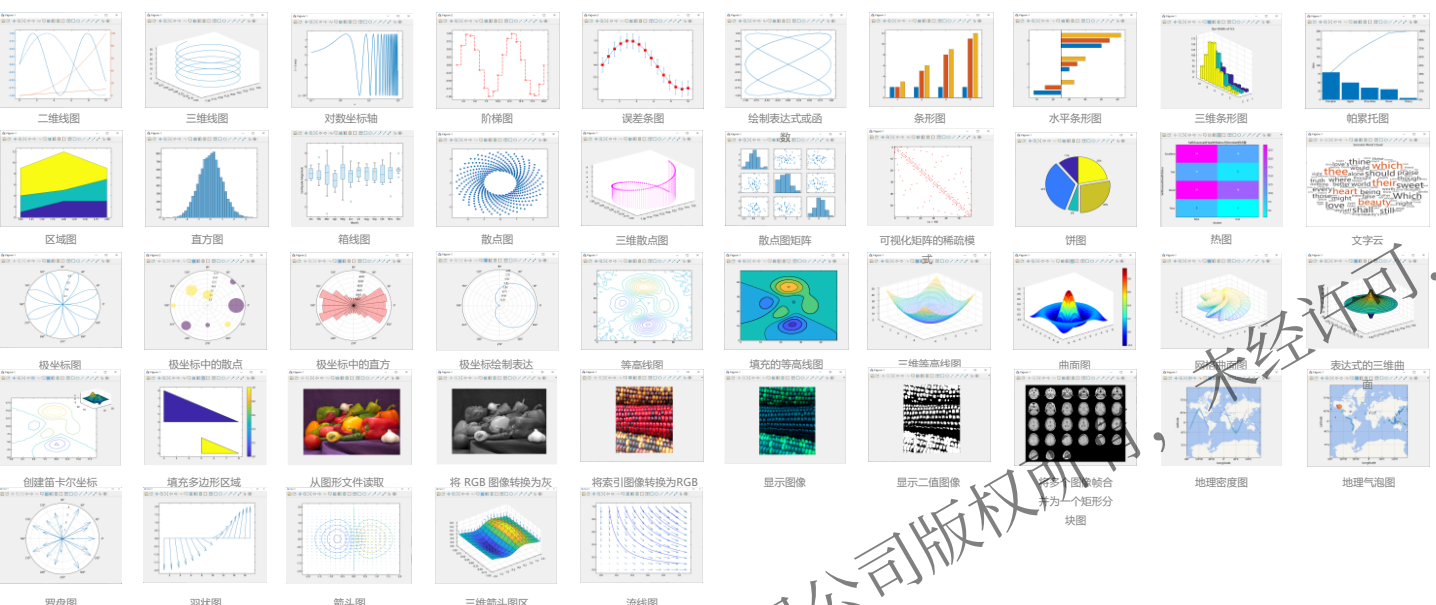
领域工具箱基础支撑环境

通过平台基础功能支撑领域工具箱的开发与运行

1.2 科学计算环境 MWORKS.Syslab-数据分析、探查与可视化

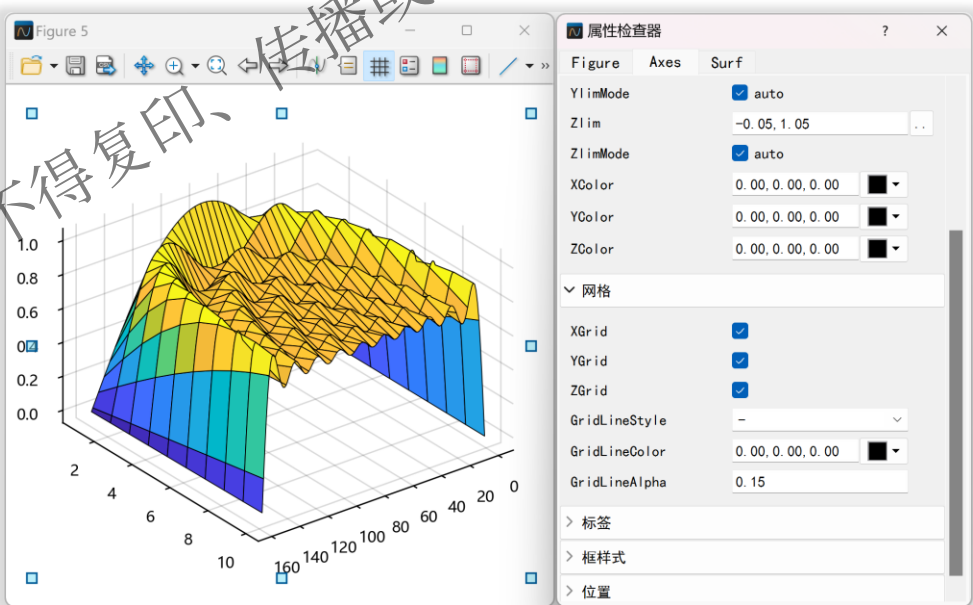
通过内置图形库创建数据可视化

超过220个绘图函数，涵盖线图、数据分布图、极坐标图、等高线图、向量场、曲面、图像、地理图等



使用交互式工具自定义可视化

易用的图窗交互功能，支持自定义绘图外观



通过Syslab绘图工具创建可视化图形



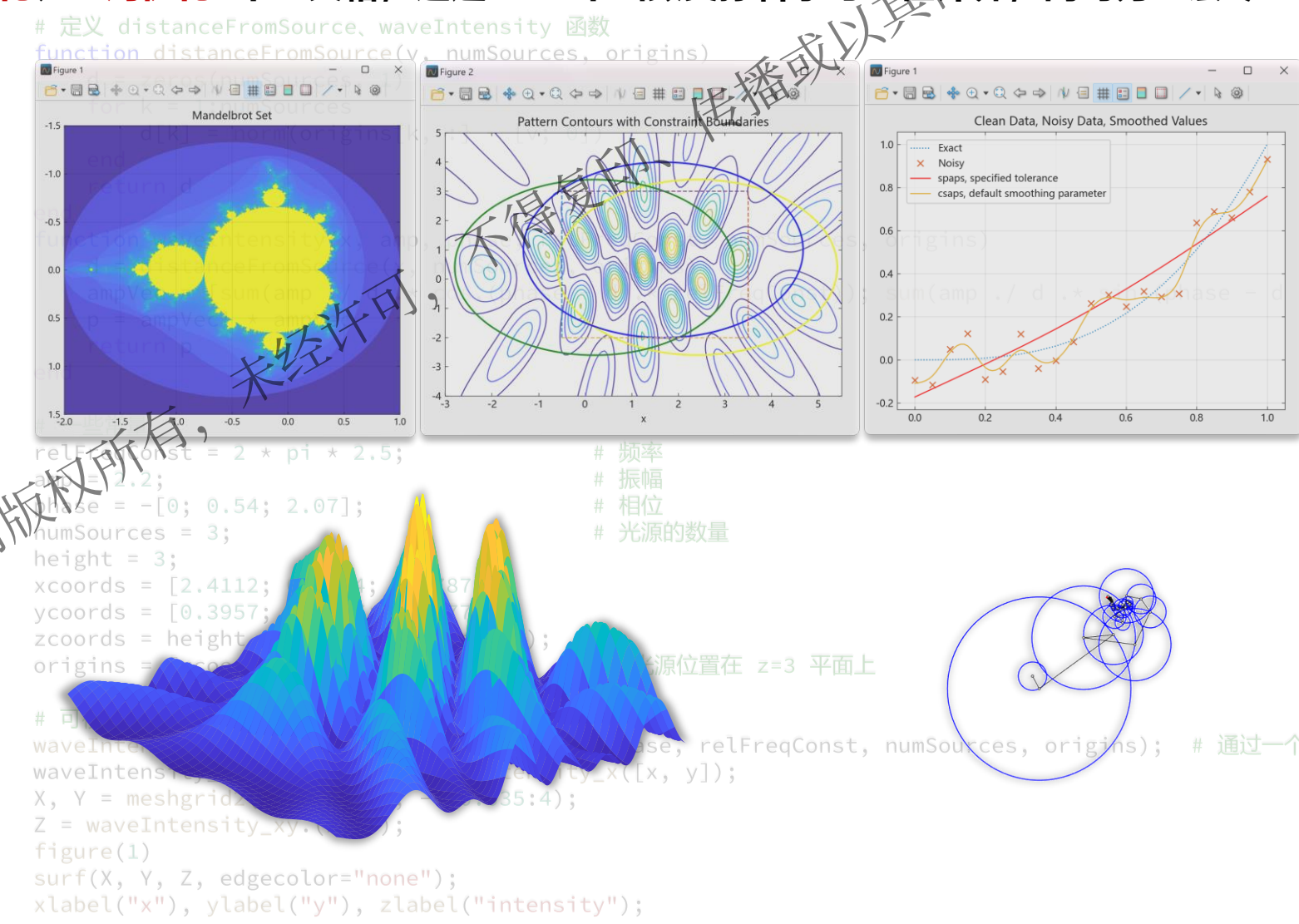
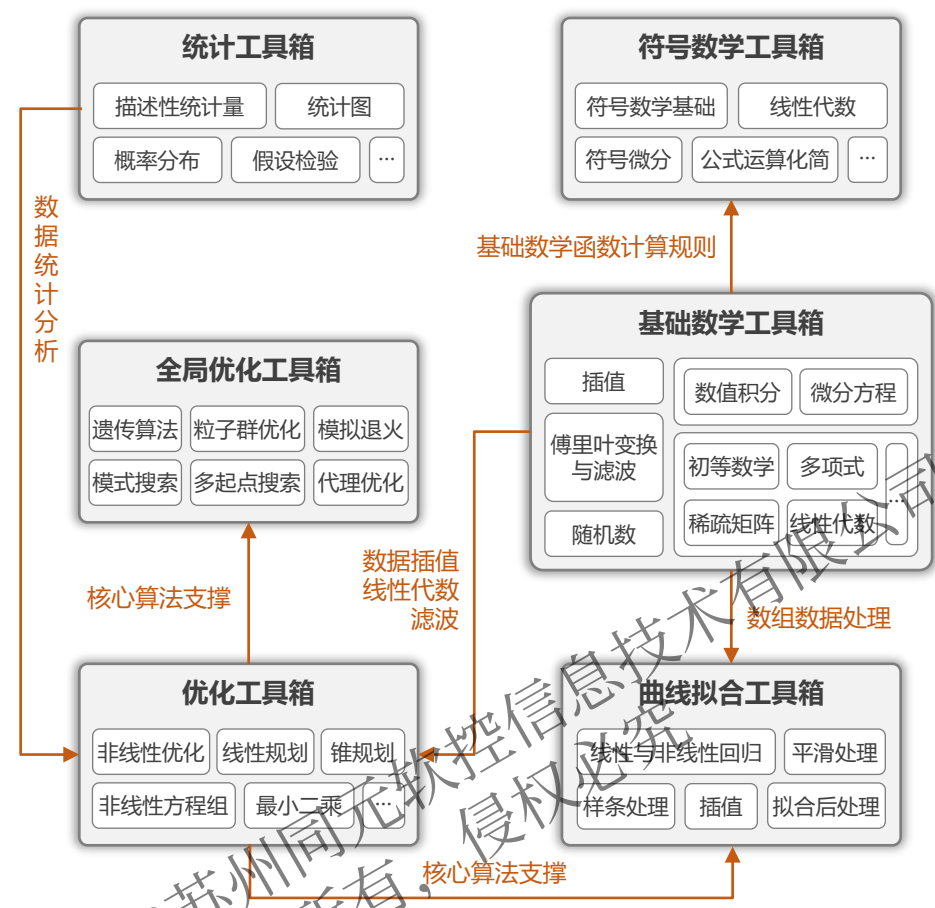
高质量图形输出

支持将绘图保存为syslabfig文件，同时支持直接导出出版级质量的图形，用于论文、海报和演示，或者保存为图像或矢量图文件，支持的格式包括：.jpg、.png、.raw、.svg、.tiff等

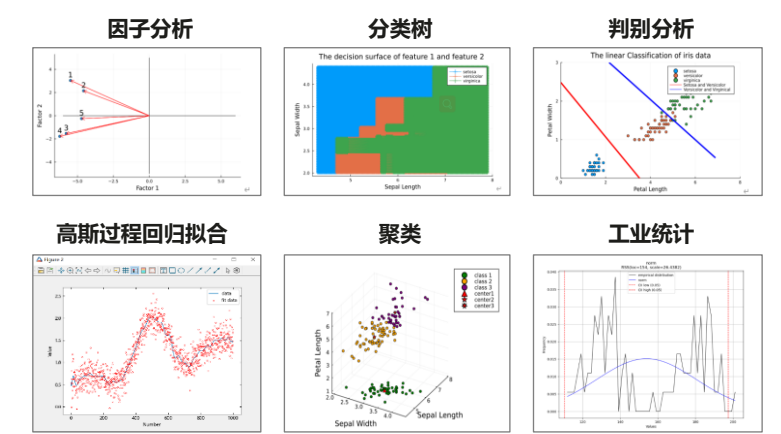
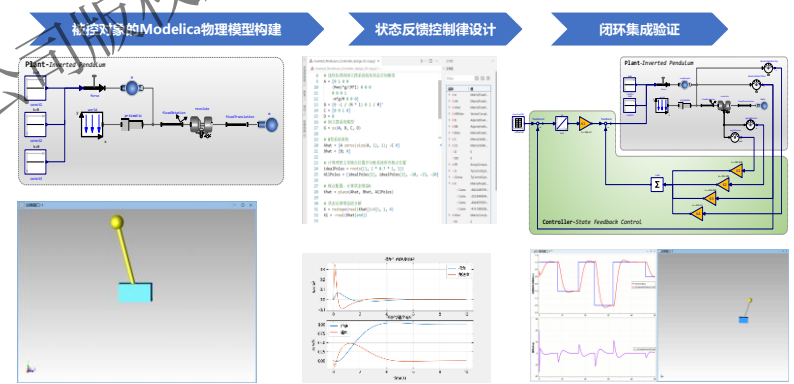
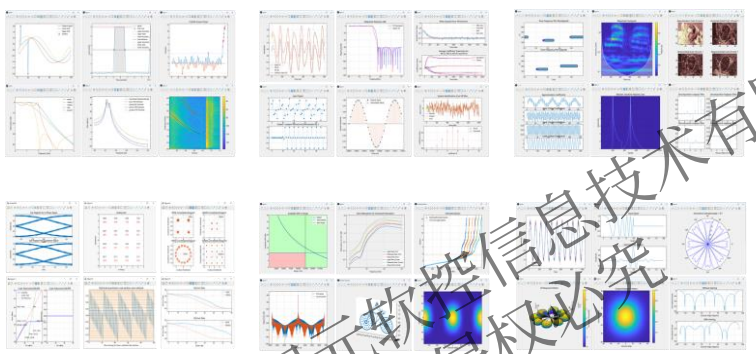
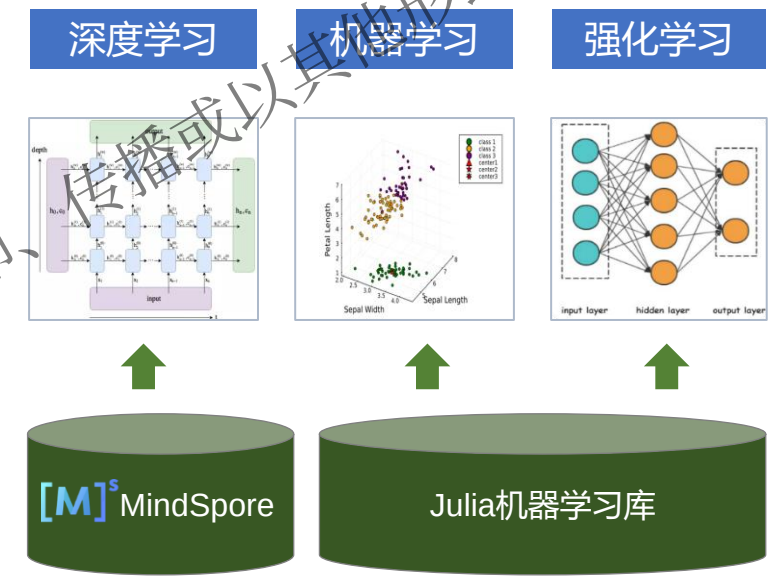
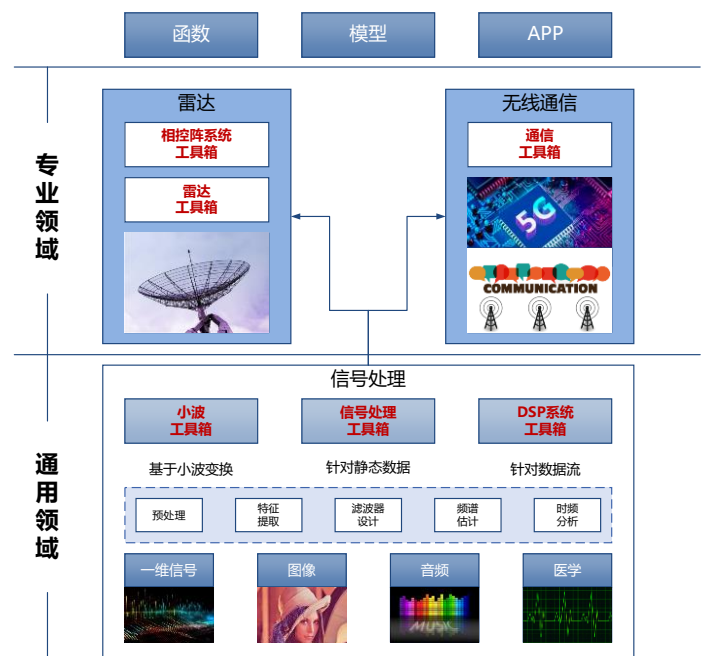


1.2 科学计算环境 MWORKS.Syslab-提供丰富高性能的数学库

通过**基础数学**、**符号数学**、**曲线拟合**、**统计**、**优化**、**全局优化**6个工具箱，超过1500个函数支撑科学与工程计算，同时为上层专业工具箱提供计算支撑



1.2 科学计算环境 MWORKS.Syslab-专业系列工具箱



信号与通信系列工具箱

控制系列工具箱

AI与数据科学系列类工具箱

1.2 科学计算环境 MWORKS.Syslab-详细的帮助手册和丰富的示例

- 丰富的综合示例，可在 Syslab 中一键打开和运行，方便用户快速学习

Syslab使用手册

目录

文档主页

示例

类别

Syslab

数学

图形

图像

统计

曲线拟合

全局优化

报告生成

文档

示例

函数

App

常微分方程

求解捕食者-猎物方程

此示例说明如何使用 ode23 和 ode45 求解表示捕食者/猎物模型的微分方程。这两个函数用于对使用变步长 Runge-Kutta 积分...

[打开示例](#)

求解抛向空中的短棒的运动方程

此示例求解常微分方程组，该方程组对抛向空中的短棒的动态进行建模。

[打开示例](#)

使用高阶求解器解决天体力学问题

此示例说明如何使用 ode78 和 ode89 解决天体力学问题，该问题需要 ODE 求解器的每一步都具有高精度才能成功积分。

[打开示例](#)

求解刚性晶体管微分代数方程

此示例说明如何使用 ode23t 求解描述电路的刚性微分代数方程 (DAE)。

[打开示例](#)

求解具有强状态依赖质量矩阵的 ODE

此示例说明如何使用移动网格方法求解 Burgers 方程。

[打开示例](#)

求解非刚性 ODE

本页包含两个使用 ode45 来求解非刚性常微分方程的示例。Syslab 提供几个非刚性 ODE 求解器。

[打开示例](#)

解算刚性 ODE

本主题说明如何将 ODE 解约束为非负解。施加非负约束不一定总是可有可无，在某些情况下，由于方程的物理解释或解性质...

[打开示例](#)

非负 ODE 解

本主题说明如何将 ODE 解约束为非负解。施加非负约束不一定总是可有可无，在某些情况下，由于方程的物理解释或解性质...

[打开示例](#)

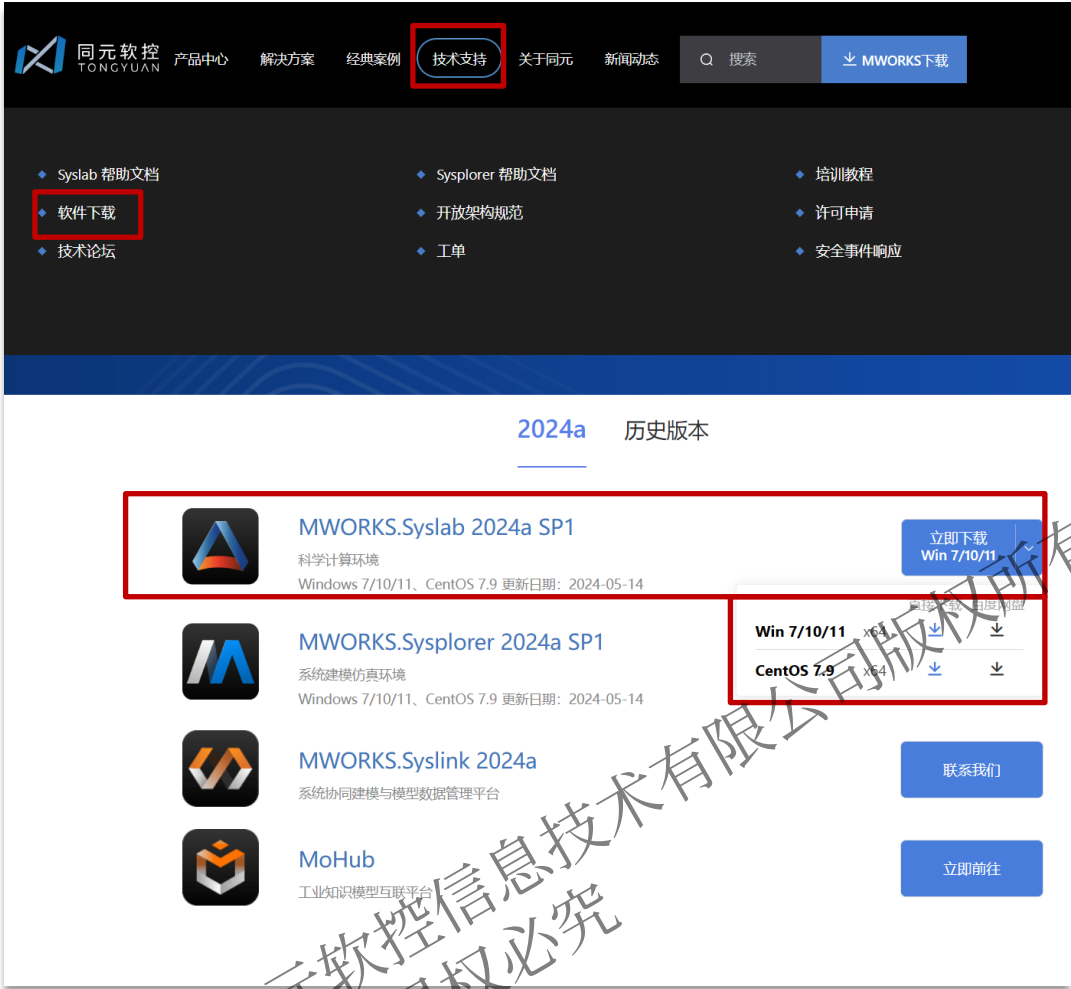
Part 2

软件安装与界面介绍

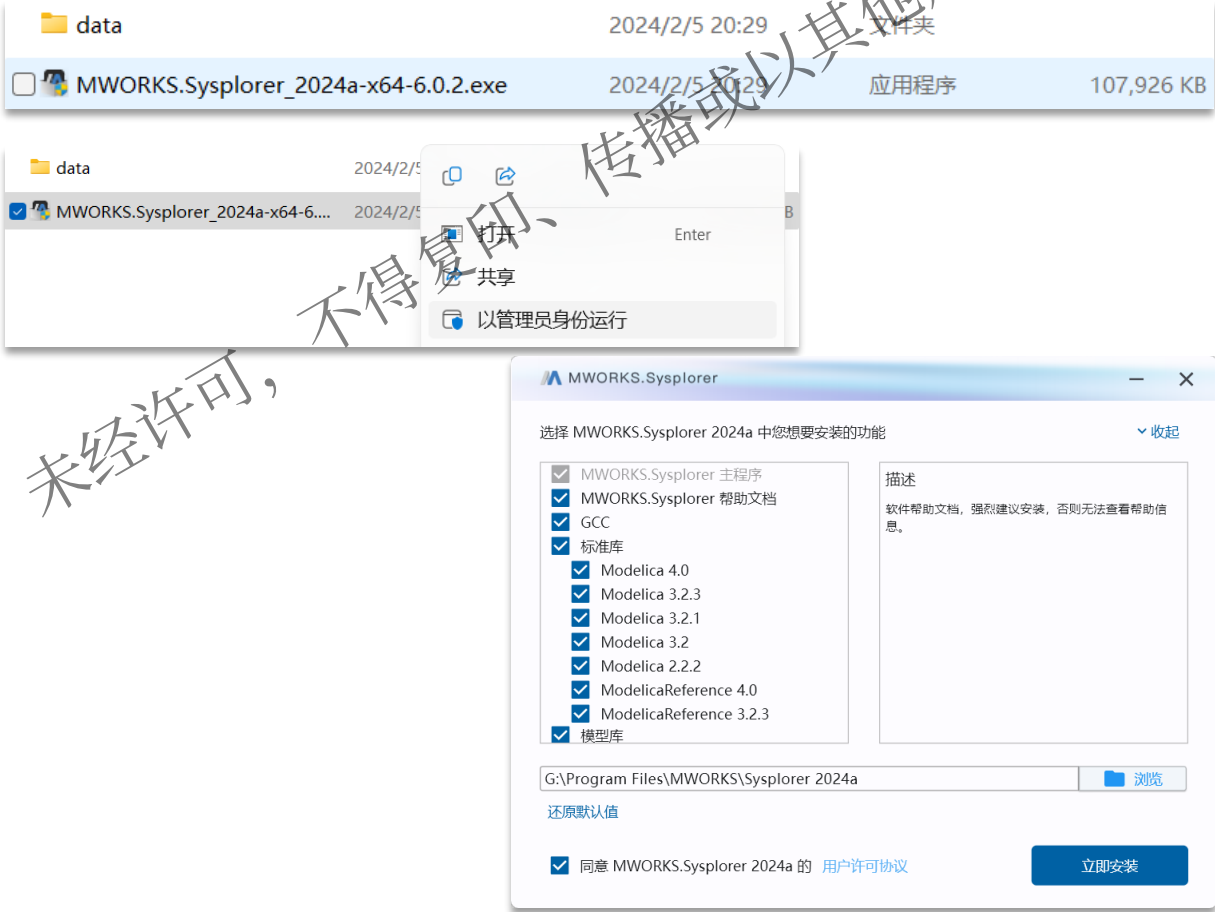
1. 软件下载与安装
2. 软件激活与配置

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

2.1 软件下载与安装

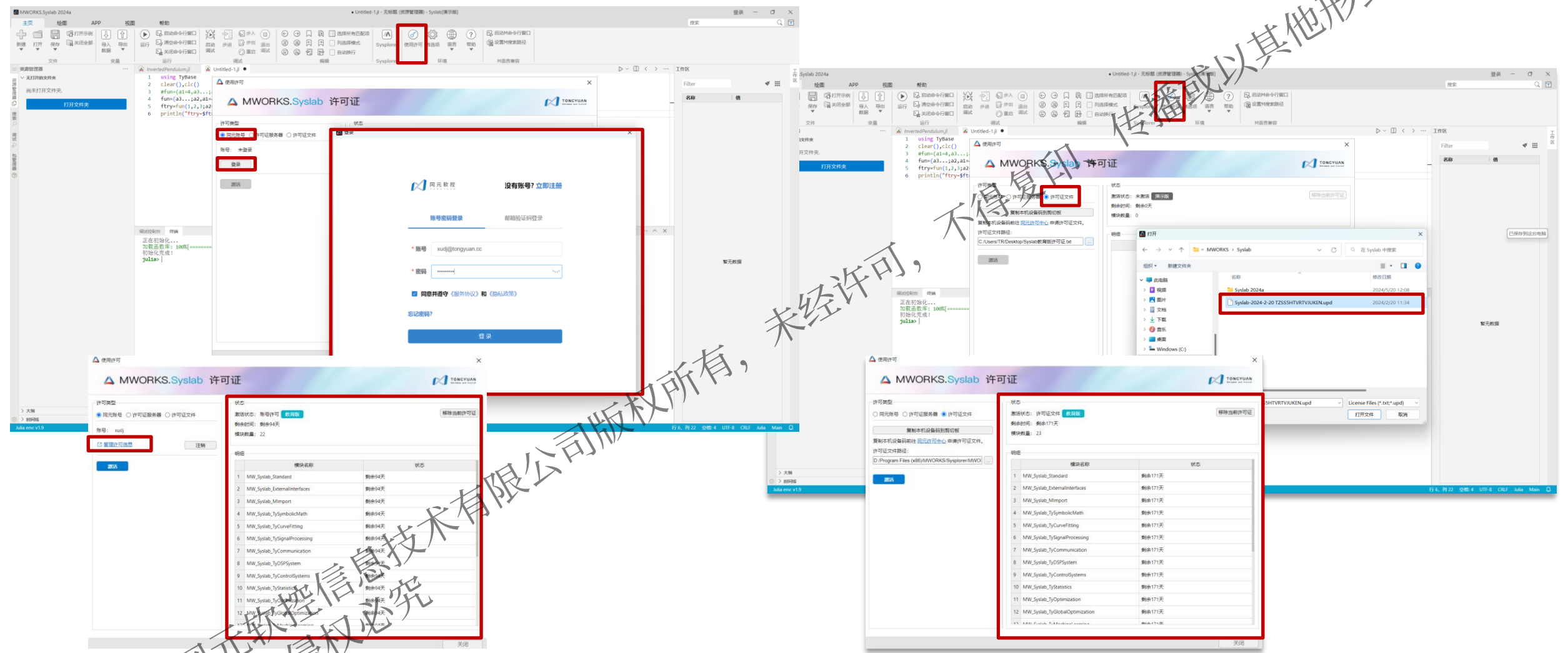


MWORKS.Syslab软件下载可登陆苏州同元软控信息技术有限公司官网
详细网址：[MWORKS-2024a-苏州同元软控\(tongyuan.cc\)](http://MWORKS-2024a-苏州同元软控(tongyuan.cc))



默认安装路径设为“C:\Program Files\MWORKS\Syslab 2023a”，如果安装在其他目录，点击浏览按钮，选择文件夹。
注：不建议软件安装路径包含中文，会出现安装失败情况

2.2 软件激活与配置

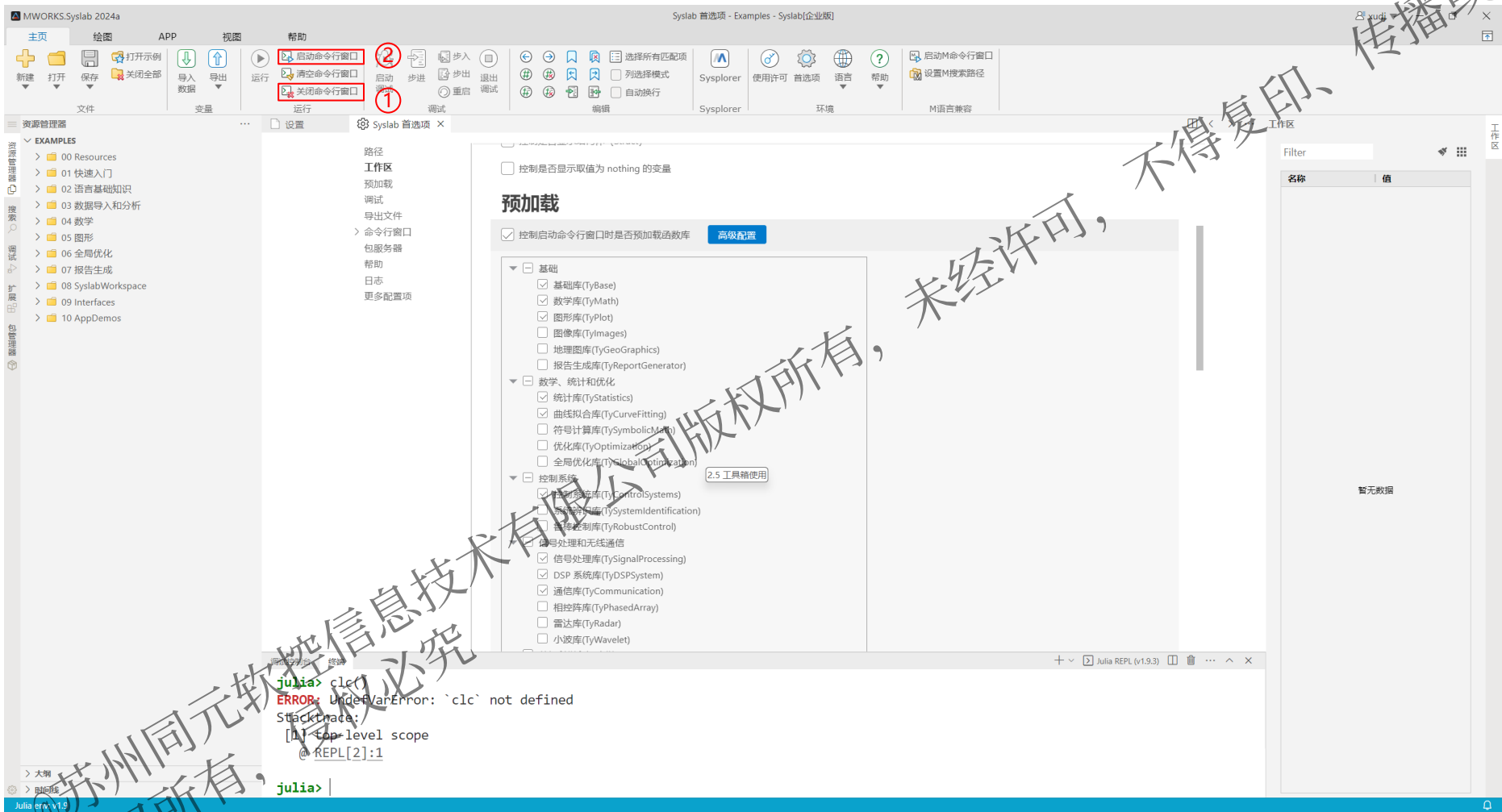


方式1：账号注册激活

方式2：许可文件配置

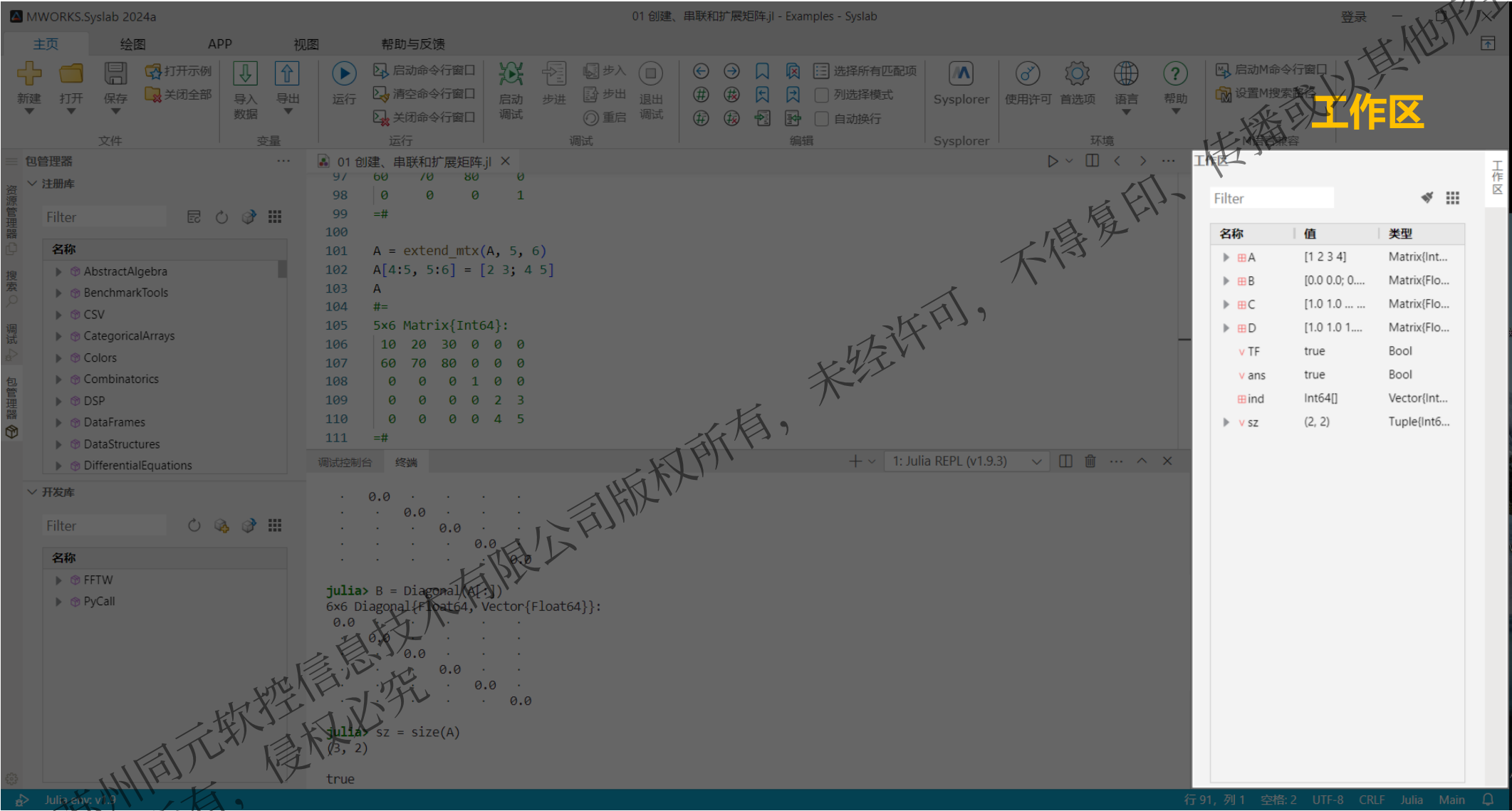
2.2 软件激活与配置

Syslab安装后**第一次打开软件默认不加载任何工具箱**，很多函数无法使用，因此我们需要点击菜单栏中的“首选项”，设置预加载同元商业工具箱，后续每次软件启动会自动加载函数库。



1. 关闭命令行窗口
2. 重新启动命令行窗口

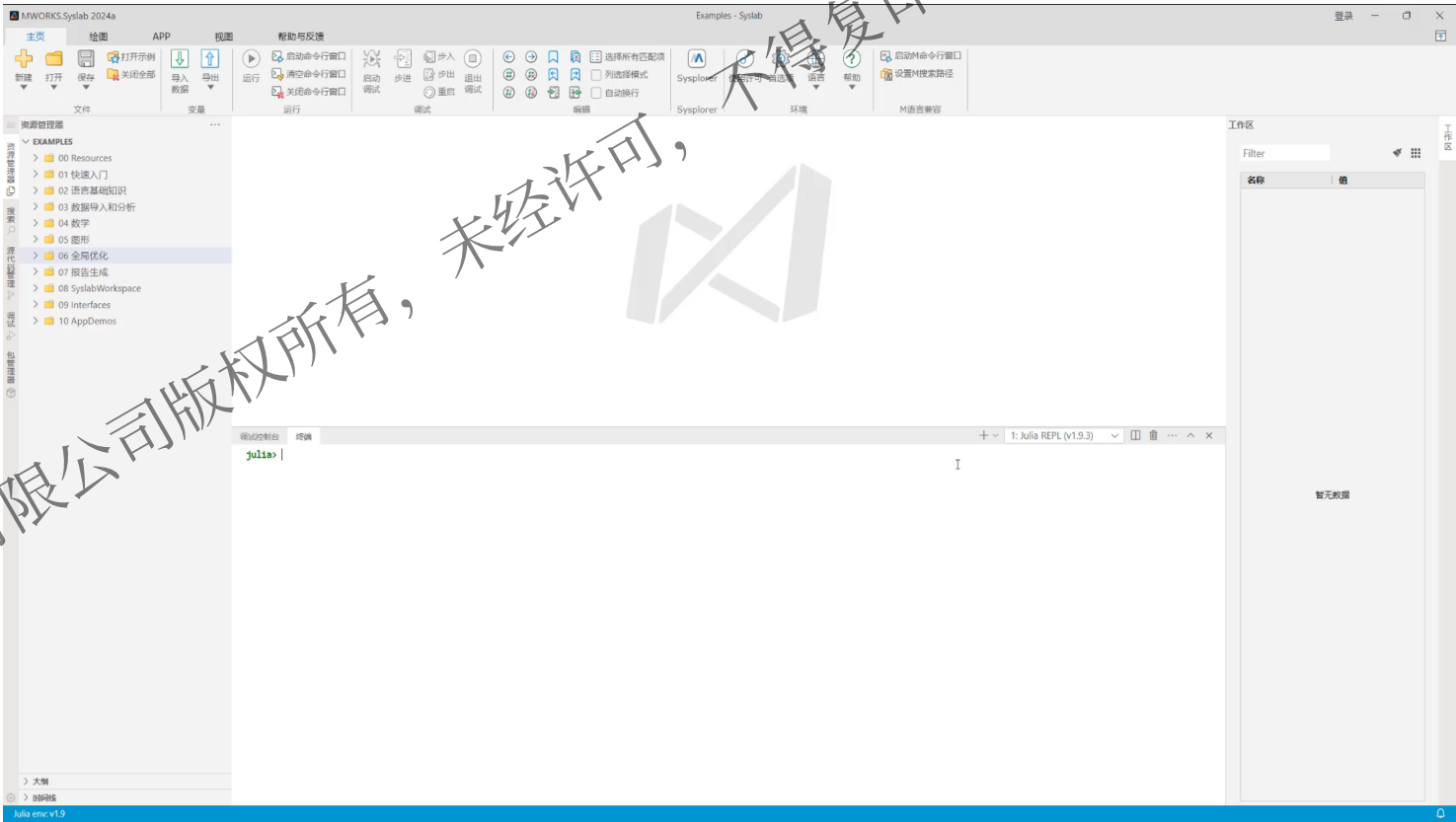
2.3 MWORKS.Syslab软件界面



2.4 MWOKRS.Syslab帮助与反馈



- 1个基础环境
- 5个行业领域
- 3300+函数帮助
- 支持分类主题
(文档、示例、函数、App)
- 支持全局搜索
- 支持在线部署
- 支持集成用户帮助文档

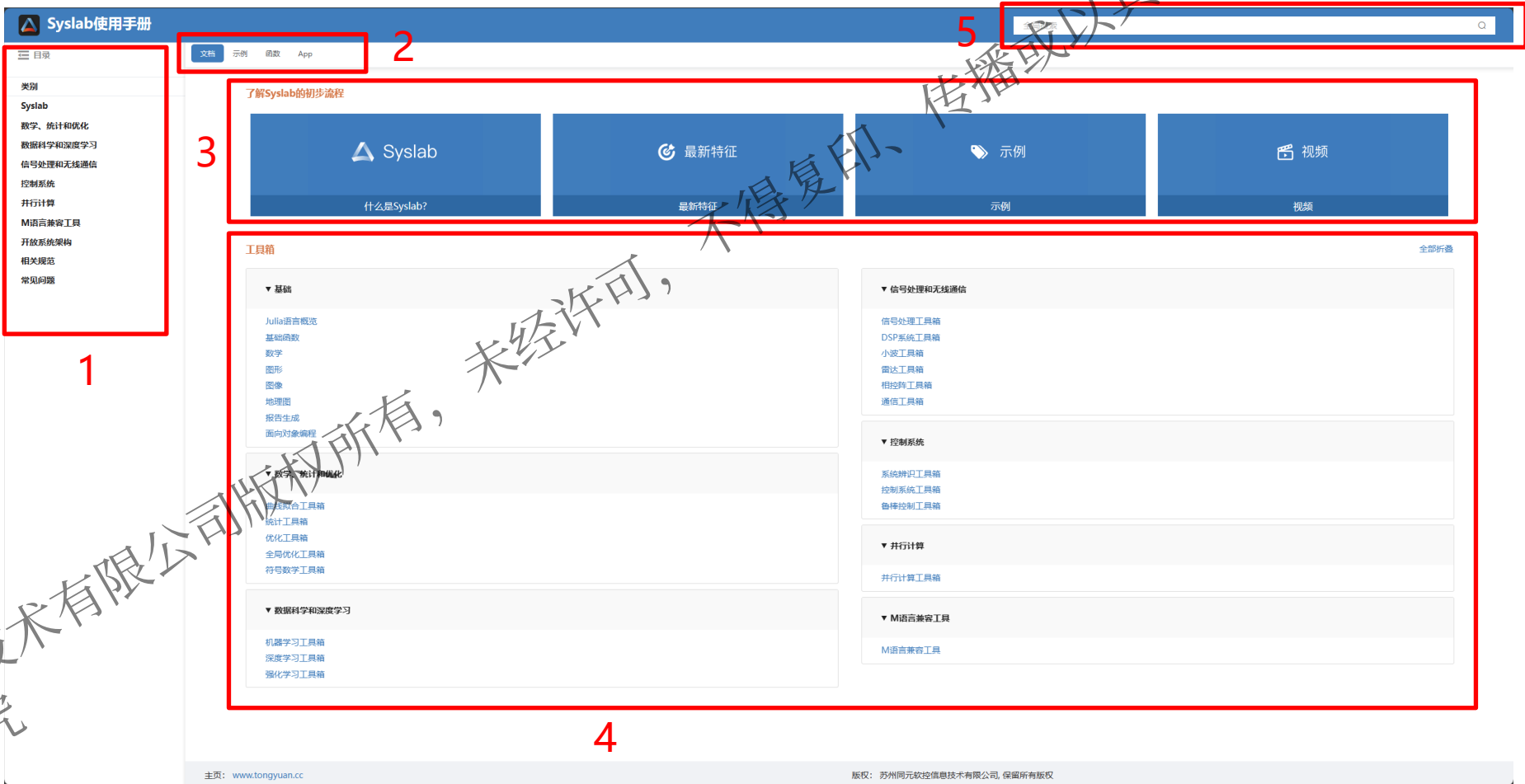


Syslab帮助中心.mp4

2.4 MWOKRS.Syslab帮助与反馈

MWORKS.Syslab帮助中心可以分为5个部分

- 1. **目录导航栏**: 提供了结构化的导航系统, 能够快速定位到感兴趣的主题或子主题。
- 2. **分类栏**: 包括文档、示例、函数、App, 可以帮助我们将查找内容类别快速分类。
- 3. **快速入门导航栏**: 包含Syslab介绍, 最新版本特征, 示例, 培训视频等连接, 帮您快速了解Syslab
- 4. **工具箱导航栏**: Syslab工具箱分类频谱, 可以快速定位感兴趣的工具箱, 进行学习或使用
- 5. **搜索栏**: 通过文字输入, 快速进行搜索查找内容



Part 3

脚本构建

1. 交互式编程环境 (REPL)
2. 文件管理
3. 脚本构建
4. 键盘快捷键

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

3.1 交互式编程环境

Syalab提供交互式编程环境，可在命令行窗口(REPL)中可直接输入命令，并得到结果。

操作步骤：

- 在Julia>后输入命令， 敲 击 “Enter” ， Syslab自 动执行并得到执行结果
- 计算2+3和sin(2)的值

```
julia> 2 + 3
5

julia> sin(2)
0.9092974268256817
```

基本命令

操作	命令
上一次运算的结果	ans
中断命令执行	Ctrl+C
清屏	Ctrl+L
运行程序文件	include("filename.jl")
查找 func 相关的帮助	?func
查找 func 的所有定义	apropos("func")
命令行模式	;
包管理模式	]
帮助模式	?
查找特殊符号输入方式	?☆ # "☆" can be typed by \bigwhitestar<tab>
退出特殊模式返回到命令行窗口	在空行上按回格键[Backspace]
退出命令行窗口	exit() 或 Ctrl+D

命令行窗口常用基本命令

命令行窗口

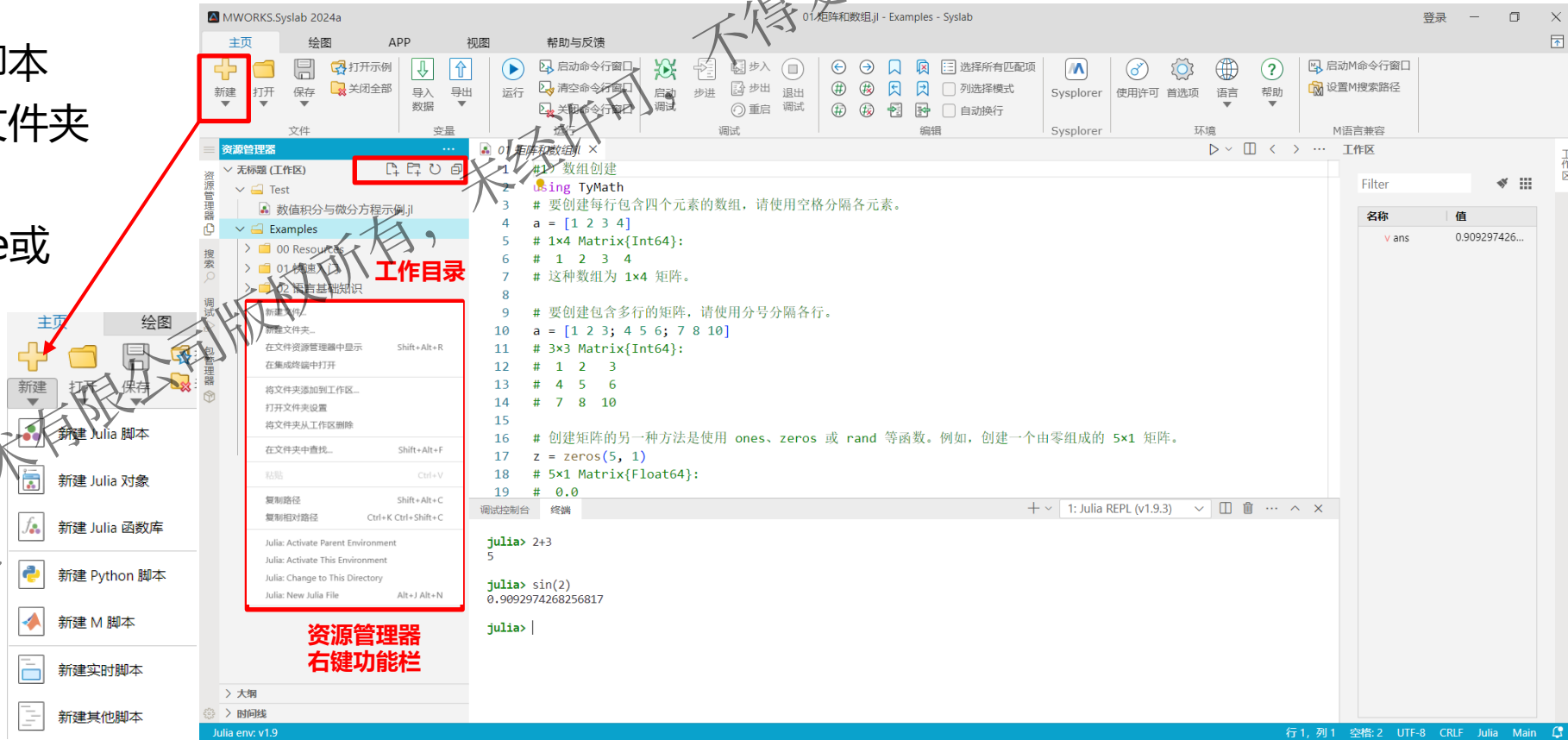


3.2 文件管理

打开相关文件夹作为工作目录，并在资源管理器中管理文件。

操作步骤:

- 选择工作目录：菜单栏中“打开”相关文件夹
- 新建脚本文件-Test
 - 工作目录下新建脚本
 - 工作目录下新建文件夹
- 对文件进行重命名
- 不需要的脚本可以delete或右击选择删除
- “新建”选择文件格式



3.3 Julia脚本构建-变量

变量是与某个值相关联（或绑定）的名字，可以用它来保存一个值（例如某些计算得到的结果），供之后的代码使用。可以直接在命令行窗口中创建变量。

例：创建变量a、b、c，分别赋值为5、10.5和“HelloWorld”，并查看变量的类型。

```
julia> a = 5
5

julia> b = 10.5
10.5

julia> c = "Hello World"
"Hello World"
```

一般形式: `x = val`

```
julia> typeof(a)
Int64

julia> typeof(b)
Float64

julia> typeof(c)
String
```

一般形式: `typeof(x)`

变量名为英文，规则如下：

- 区分大小写；
- 不能以数字开头；
- 变量与函数名建议用下划线分隔；
- 类与模块首字母建议大写，驼峰式；

变量的数据类型由赋值决定

3.3 Julia脚本构建-变量

变量不只可以为英文，还可以是中文或特殊字符

例：创建变量“变量1”，值为3.14，并查看变量的值。

```
julia> 变量1 = 3.14
3.14

julia> 变量1
3.14
```

例：创建变量“β”，值为36，并查看变量的值。

```
julia> β = 36 #\beta,再敲击Tab键
36

julia> β
36
```

Unicode字符使用方式：
输入某个LaTeX符号（比如\beta），再敲击Tab键。

LaTeX符号详见：《Julia 中文文档》- Unicode输入表



3.3 Julia脚本构建-数值类型

整数类型:

类型	是否带符号	比特数	最小值	最大值
Int8	√	8	-2^7	2^7-1
UInt8		8	0	2^8-1
Int16	√	16	-2^{15}	$2^{15}-1$
UInt16		16	0	$2^{16}-1$
Int32	√	32	-2^{31}	$2^{31}-1$
UInt32		32	0	$2^{32}-1$
Int64	√	64	-2^{63}	$2^{63}-1$
UInt64		64	0	$2^{64}-1$
Int128	√	128	-2^{127}	$2^{127}-1$
UInt128		128	0	$2^{128}-1$
Bool	N/A	8	false(0)	true(1)

```
julia> a = 1
1

julia> typeof(a) #操作系统为64位
Int64

julia> max = typemax{Int64} #整数最大值
9223372036854775807

julia> max + 1 #超限
-9223372036854775808

julia> min = typemin{Int64} #整数最小值
-9223372036854775808

julia> min - 1 #超限
9223372036854775807

julia> x = typeof(0x123)
UInt16

julia> Int64(0x123) #转化为Int64
291
```

- 说明:
- 根据操作系统不同，整数 Int 可能是 Int32 或 Int64
 - 超出一个类型可表示的范围会导致环绕
 - 无符号整数会使用0x为前缀的十六进制来表示
 - 可以使用convert(类型,x)进行数据类型转换



3.3 Julia脚本构建-数值类型

浮点类型：用于表示小数

类型	精度	比特数
Float16	半(half)	16
Float32	单(single)	32
Float64	双(double)	64

```
julia> a = 5.2
5.2

julia> typeof(a) #操作系统为64位
Float64

julia> typeof(1e5) #操作系统为64位
Float64

julia> typeof(5.2f0) #使用f则为32位
Float32

julia> Float32(a)
5.2f0
```

说明：

- 浮点数的默认类型取决于电脑系统是32位还是64位
- 浮点数可以用科学记数法表示
- 使用“f”替代“e”或Float32()可以得到Float32的浮点数
- 半精度一般采用软件模拟，性能较差，不推荐使用



3.3 Julia脚本构建-分数

分数的构建

一般形式: `a // b`

分数通过 “//” 构建，用于表示整数精确比值的分数类型

```
julia> 2//3
2//3
```

注意:
分子分母只能为**整型**。

分数的标准化

如果一个分数的分子和分母含有公因子，它们会被约分到最简形式且分母非负：

```
julia> 6//9
2//3

julia> -4//8
-1//2

julia> 5//-15
-1//3

julia> -4//-12
1//3
```

分数的标准化分子和分母分别可以使用 `numerator` 和 `denominator` 函数得到：

```
julia> numerator(4 // 6) #查看标准化分子
2

julia> numerator(2//3)
2

julia> denominator(4//6) #查看标准化分母
3

julia> denominator(2//3)
3
```



3.3 Julia脚本构建-复数

复数的定义

一般形式: `a + b* im`

Julia中复数的虚部用 “im” 表示, im为全局变量。

注意:
不能使用i和j来表示虚部

复数的构建

方法一(im直接构建):

```
Julia> 1 + 2im
1 + 2im

julia> 1+Inf*im
1.0 + Inf*im

julia> 1+NaN*im
1.0 + NaN*im
```

注意:
`a+bim`, `b`如果为变量名称时则错误, 此时必须是`a+b*im`

方法二(complex函数构建):

```
julia> a=1;b=2;complex(a,b)
1 + 2im
```

3.3 Julia脚本构建-数值运算

Syslab中提供了大量数值运算符，可以直接使用

类型	表达式	名称	描述
算数运算符	+x	一元加法	全等操作
	-x	一元减法	将值变为其相反数
	$\sqrt{x}$	平方根	求解平方根(\sqrt{x})
	x + y	二元加法	执行加法
	x - y	二元减法	执行减法
	x * y	乘法	执行乘法
	x / y	除法	执行除法
	x $\div$ y	整除	取x/y的整数部分(\div)
	x \ y	反向除法	等价于y/x
	x ^ y	幂运算	x的y次幂
	x % y	取余	等价于rem(x,y)



3.3 Julia脚本构建-数组

向量定义

```
julia> a = [1,2,3]
3-element Vector{Int64}:
 1
 2
 3
```

一般形式: **x = [x1,x2,x3]**

使用 **,** 分隔向量的元素

```
julia> b = [1;2;3]
3-element Vector{Int64}:
 1
 2
 3
```

一般形式: **x = [x1;x2;x3]**

使用 **;** 分隔向量的元素

矩阵定义

```
julia> c = [1 2;3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

一般形式: **x = [x1 x2;x3 x4]**

使用 **空格** 分隔矩阵列, 使用 **;** 分隔矩阵行

```
julia> c = [1 2
           3 4]
2×2 Matrix{Int64}:
 1  2
 3  4
```

一般形式: **x = [x1 x2
x3 x4]**

使用 **空格** 分隔矩阵列, 使用 **Alt+Enter** 分隔矩阵行



3.3 Julia脚本构建-数组

函数	功能
ones	生成全1元素数组
zeros	生成全0元素数组
eye	生成单位矩阵
rand	生成均匀分布随机矩阵
randn	生成正态分布随机矩阵
magic	生成魔方矩阵，即矩阵各行各列的和都相等
Diagonal	生成对角矩阵

特殊数组内置函数

构建向量a[3]，其中a每个元素均为1

```
julia> a = ones(3)
3-element Vector{Float64}:
 1.0
 1.0
 1.0
```

构建3阶单位矩阵b

```
julia> b = eye(3)
3×3 Matrix{Int64}:
 1 0 0
 0 1 0
 0 0 1
```

```
julia> b = eye(3,3)
3×3 Matrix{Int64}:
 1 0 0
 0 1 0
 0 0 1
```



3.3 Julia脚本构建-数组

向量拼接为矩阵

定义3阶0向量a,
定义3阶1向量b,
将a向量和b向量拼接形成矩阵A

```
julia> a = zeros(3);  
  
julia> b = ones(3);  
  
julia> A = [a b]  
3×2 Matrix{Float64}:  
0.0 1.0  
0.0 1.0  
0.0 1.0
```

矩阵拼接为矩阵

将3阶魔方矩阵A拼接形成矩阵B:
$$\begin{bmatrix} A & \text{zeros}(3,1) \\ \text{zeros}(1,3) & 1 \end{bmatrix}$$

```
julia> A = magic(3);  
  
julia> B = [A zeros(3,1);zeros(1,3) 1 ]  
4×4 Matrix{Float64}:  
8.0 1.0 6.0 0.0  
3.0 5.0 7.0 0.0  
4.0 9.0 2.0 0.0  
0.0 0.0 0.0 1.0
```

- 分号表示纵向(行, 第一维度)拼接
- 空格表示横向(列, 第二维度)拼接
- 空格的优先级高于分号, 即先横向拼接, 再纵向拼接



3.3 Julia脚本构建-数组

定义3阶魔方矩阵A,
访问A的第1行, 第2列的元素,
访问A的第1行所有元素,
访问A的第2列所有元素,
访问A的第1行至第2行, 第2列到第3列所有元素,
访问A的第2行到最后1行的所有元素。



访问A的第m行第n列的元素, $a = A[m,n]$
访问A的第m行的所有元素, $b = A[m,:]$
访问A的第n列的所有元素, $c = A[:,n]$
访问A的第n行到第m行, 第n列到第m列元素, $d = A[n:m,n:m]$
访问A的第m行到最后一行的所有元素, $e = A[m:end,:]$

```
julia> A = magic(3)
3×3 Matrix{Int64}:
 8  1  6
 3  5  7
 4  9  2

julia> B = A[1,2]
1
```

```
julia> C = A[1,:]
3-element Vector{Int64}:
 8
 1
 6

julia> D = A[:,2]
3-element Vector{Int64}:
 1
 5
 9
```

```
julia> E = A[1:2,2:3]
2×2 Matrix{Int64}:
 1  6
 5  7

julia> F = A[2:end,:]
2×3 Matrix{Int64}:
 3  5  7
 4  9  2
```



3.3 Julia脚本构建-数组

数组可以通过索引与切片使用或修改数组

一般形式： Julia数组使用方括号A[]进行索引和切片。

```
X = A[l_1,l_2,...,l_n]
```

说明：

- l_n可以为标量整数、整数数组或其他支持的索引类型
- 数组索引：l_n均为为标量整数时，结果X为数组A中的单个元素
- 数组切片：l_n不全为标量整数时，结果X为数组A中的一个区域数组

```
julia> A = reshape(collect(1:4),(2,2))
2×2 Matrix{Int64}:
 1  3
 2  4
julia> A[1,2];A[3] #数组元素位置从1开始; 支持线性索引
 3
julia> A[1,[1,2]] # [1,2]表示此维度中第1个和第2个量
2-element Vector{Int64}:
 1
 3
julia> A[[1],[1,2]]
1×2 Matrix{Int64}:
 1  3
```

注意：

- 取矩阵中某一行或一列中某一个或某几个值时，取出的值为对应个数的**向量**。

```
julia> A[1,:] #使用 取维度中所有量
2-element Vector{Int64}:
 1
 3
julia> A[1,1:2] # 1:2表示此维度中第1个到第2个量
2-element Vector{Int64}:
 1
 3
julia> A[end,2] #end表示此维度中最后1个量
 4
julia> A[[false,true],:] # [false,true]表示此维度中对应位置为true的量
1×2 Matrix{Int64}:
 2  4
julia> A[CartesianIndex(2, 1)] #CartesianIndex(2, 1) 表示使用笛卡尔坐标索引
 2
```



3.3 Julia脚本构建-数组

注意: Julia 不会在赋值语句中自动增长数组。

```
julia> a = rand(3,3)
3×3 Matrix{Float64}:
 0.317134  0.808967  0.912479
 0.241704  0.420218  0.245492
 0.00407314 0.447554 0.506671

julia> a[3,1] #索引第3行第1列的值
0.0040731381120721055

julia> a[4,1] #索引第4行第1列的值不存在
ERROR: BoundsError: attempt to access 3×3 Matrix{Float64} at index [4, 1]
Stacktrace:
 [1] getindex(::Matrix{Float64}, ::Int64, ::Int64)
       @ Base .\array.jl:862
 [2] top-level scope
       @ REPL[3]:1
```

Julia内提供了push! 和append!(a,b)

```
julia> A=[1,2,3]
3-element Vector{Int64}:
 1
 2
 3
#将值作为最后一个数压入数组
julia> push!(A, 200)
4-element Vector{Int64}:
 1
 2
 3
 200
b=[100,200]
#append!要求相同维度
append!(A,b)
```

3.3 Julia脚本构建-数组

常用运算符与函数

类型	表达式	名称	描述
一元运算符	+	一元加法	全等操作
	-	一元减法	将各元素取反
二元运算符	-	减	各元素相减
	+	加	各元素相减
	.*	点乘	各元素相乘
	./	点除	各元素相除
	*	乘	数组乘法（线性代数）
	/	除	数组除法（线性代数）
	\	反向除法	数组反向除法（线性代数）
	^	幂	数组幂运算
常用函数	det	矩阵行列式	计算矩阵行列式
	rank	矩阵的秩	计算矩阵的秩
	inv	矩阵求逆	计算矩阵的逆
	eigvals	矩阵的特征值	计算矩阵的特征值
	transpose	向量或矩阵转置	将向量或矩阵进行转置



3.3 Julia脚本构建-数组

注意：Julia 数组在分配给另一个变量时不会被复制。
在A = B之后，改变B的元素也会改变A的元素。

```
julia> A=[1,2,3]
3-element Vector{Int64}:
 1
 2
 3

julia> B=A
3-element Vector{Int64}:
 1
 2
 3

julia> A[1]=2
2

julia> B
3-element Vector{Int64}:
 2
 2
 3

pointer(A)==pointer(B)
true

julia> c=copy(A)
3-element Vector{Int64}:
 2
 2
 3

julia> A[3]=2
2

julia> c
3-element Vector{Int64}:
 2
 2
 3

pointer(A)==pointer(c)
false
```

数组切片：默认情况是复制内存的

```
julia> X = [1, 2, 3, 4]
4-element Vector{Int64}:
 1
 2
 3
 4

julia> Y = X[1:2] #Y取X的第1个和第2个值
2-element Vector{Int64}:
 1
 2

julia> Y[1] = 0 #改变Y的第一个值
0

julia> X[1] #X的第一个值依然为1
1
```



3.3 Julia脚本构建-函数

当内置函数不能满足需求时，可根据需求自定义函数，并进行使用

定义函数 $f(x_1, x_2) = x_1^2 + x_2^2 - 2x_1x_2 + 6x_1 - 6x_2$ 并求解 $f(1,2)$ 的值



新建脚本文件Myfun.jl

```
function myfun(x1,x2)
    z = x1^2-2x1*x2+6x1+x2^2-6x2;
end
```

定义函数myfun
x1和x2为形参，z为返回值

```
myfun(x1, x2) = x1^2 - 2x1 * x2 + 6x1 + x2^2 - 6x2
```

简写定义函数myfun
x1和x2为形参

```
myfun (generic function with 1 method)
```

运行函数定义

```
include("Myfun.jl")
a = myfun(1,2)
```

脚本中直接调用函数脚本
include("函数脚本路径")

```
julia> a = myfun(1,2)
-5
```

函数调用

3.3 Julia脚本构建-函数

向量化调用-广播 ☆ ☆ ☆

任何 Julia 函数 f 能够以元素方式作用于任何数组（或者其它集合），这通过语法 f(A) 实现。

计算 $z_2 = \frac{1}{2} \ln(x + \sqrt{1+x^2})$, 其中 $x = \begin{bmatrix} 2 & 1+2i \\ -0.45 & 5 \end{bmatrix}$

```
julia> x = [2 1+2im;-0.45 5]
2×2 Matrix{ComplexF64}:
 2.0+0.0im  1.0+2.0im
-0.45+0.0im  5.0+0.0im

julia> z2=1/2*log.(x+sqrt.(1.+x.^2))
2×2 Matrix{ComplexF64}:
0.711397-0.0253138im  0.896799+0.365773im
0.21387+0.934284im   1.15408-0.00442594im
```

sqrt.(A)即表示对数组A中的每一个元素进行求平方根

重要提示：任何 Julia对于数组的操作都建议使用广播
任何 Julia对于数组的操作都建议使用广播
任何 Julia对于数组的操作都建议使用广播



3.3 Julia脚本构建-函数

广播(broadcast): 将数组参数中低维度的参数扩展，使得其与其他维度匹配，且不会使用额外的内存，并将所给的函数逐元素地应用。

一般形式: f(args...)或运算符

```
julia> a = rand(2, 1); A = rand(2, 3);

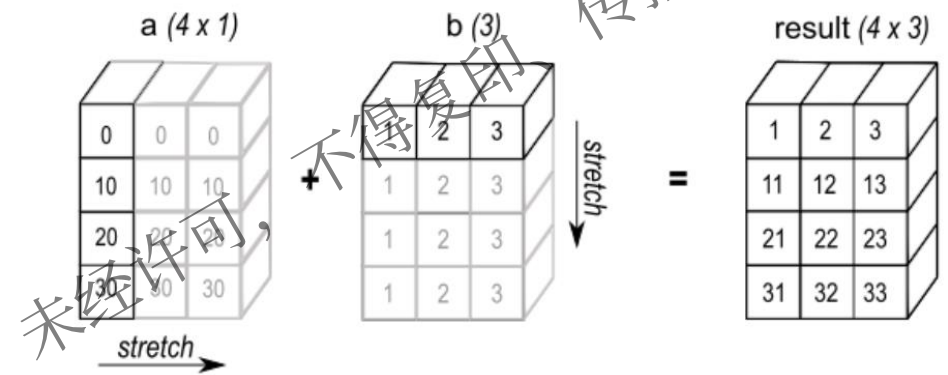
#repeat将a重复形成2×3的矩阵后与A相加
julia> repeat(a, 1, 3) + A
2×3 Matrix{Float64}:
 1.01136  1.16664  1.63508
 0.837383 1.49359  0.599233
```

当repeat(a, 1, 3)与A维度相同时可以使用 “+” 直接相加，a与A维度不同，直接相加会出现什么结果呢？

```
julia> a + A
ERROR: DimensionMismatch: dimensions
must match: a has dims (Base.OneTo(2),
Base.OneTo(1)), b has dims
(Base.OneTo(2), Base.OneTo(3)),
mismatch at 2
```

报错了！

从上面结果我们知道，数组相加过程不会自动扩展维度，广播的方式可以帮我们很好的解决此类问题



```
julia> c=convert(Float64,1)
1.0

julia> C=convert.(Float64,[1 2]) #使用广播的形式转换矩阵
1×2 Matrix{Float64}:
 1.0 2.0

julia> D=convert.(Float64, (1,2)) #处理元组
(1.0, 2.0)
```

说明:

- 广播不限于数组, 还可以处理标量、元组和其它容器
- 推荐尽量使用广播的形式，广播能够提高性能



3.3 Julia脚本构建-函数

Syslab中集成了大量内置函数，可根据需求直接使用，并提供了丰富的中文帮助说明

计算sin(2)，cos(3)，并取sin(2)和cos(3)的最大值和最小值

```
julia> a = sin(2)
0.9092974268256817

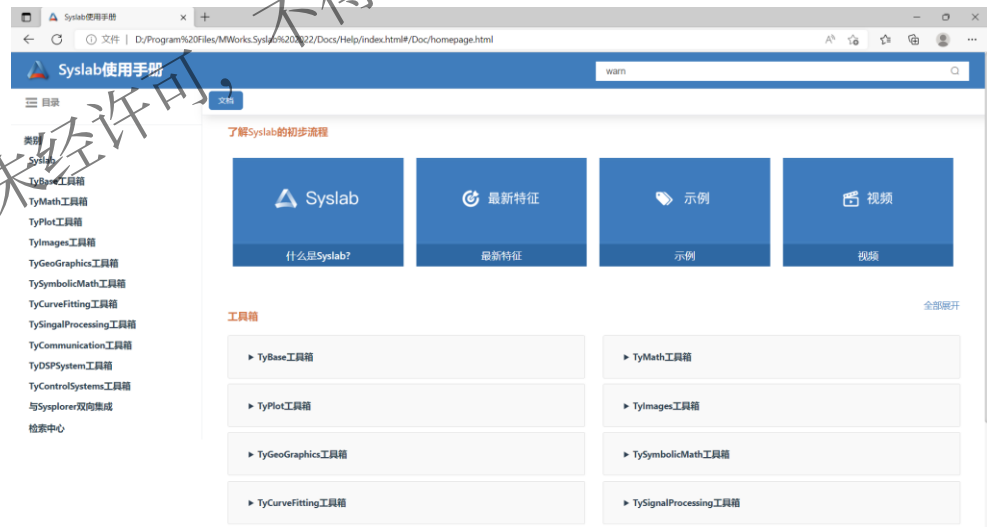
julia> b = cos(3)
-0.9899924966004454

julia> c = max(a,b)
0.9092974268256817

julia> d = min(a,b)
-0.9899924966004454
```

使用形式：**x = func(a,b,c)**

内置函数的功能、使用方式、示例均提供帮助说明
方式1：可查看**Syslab帮助中心**



方式2：
Shift+? 进入**help**模式
输入函数名即可获得帮助说明

```
help?> plot
search: plot plot3 plotmatrix plot_orb

plot - 二维线图

此 Syslab 函数 创建 Y 中数据对 X 中 X
则它们的大小必须相同。plot 函数绘制 Y
中的一个为向量而另一个是矩阵，则矩阵
是或向量。则 plot 函数会绘制离散点。但
```



3.3 Julia脚本构建-函数

输入命令
编译和运行 Syslab 语句
在 Syslab 中工作时，您
可以发出创建变量和调用
函数的命令。

函数名	简介
ans	最近计算的答案
clc()	清空命令行窗口
clear()	清空工作区内容
clipboard()	在目标与系统剪贴板之间复制和粘贴文本
Ctrl + D	终止 Syslab 程序 (与 exit() 等效)
exit()	终止 Syslab 程序
gethostname()	获取本地计算机的主机名
import	导入整个库或者库中的模块
using	加载整个库或者库中的模块，并使其可直接使用
varinfo()	展示模块中的变量及大小和类型
VersionNumber(str)	用于将字符串解析为版本号
@show	显示表达式和结果，返回结果
@time	用于执行表达式的宏，打印执行所需的时间、分配数及其字节总数



3.3 Julia脚本构建-控制流

一般格式:

```
if 条件表达式1
    #执行语句
elseif 条件表达式2
    #执行语句
elseif 条件表达式3
    #执行语句
else
    #执行语句
end
```

注意:

- 条件表达式中if是必须的，且只能出现一次；
- else可以不出现但若出现则只能一次；
- elseif可多次使用；
- 条件表达式与关键词之间用空格隔开
- 条件表达式为布尔量

```
julia> if 3>0
3
elseif 2>0
2
end
3

julia> if 1
println("true")
end
ERROR: TypeError: non-boolean (Int64) used in boolean context

julia> z = if 3 < 0
10
else
-10
end
-10
```



3.3 Julia脚本构建-控制流

一般格式:

```
while 条件表达式
    #执行语句
end
```

```
julia> i=1
1

julia> while i <= 5
    println(i)
    i += 1
end

1
2
3
4
5

julia> i
6
```

说明:

- 条件表达式为布尔量，为true时，执行语句才会执行;



3.3 Julia脚本构建-控制流

for语句:

for 循环使用起来会更加方便，以上实例使用 for 循环实现如下：

```
julia> for i = 1:5
    print(i)
end
12345
```

这里的 1:5 是一个范围对象，代表数字 1, 2, 3, 4, 5 的序列。
for 循环在这些值之中迭代，对每一个变量 i 进行赋值。

一般格式(单个迭代变量):

```
格式1:
    for 迭代变量=可迭代数集
        #语句块
    end
格式2:
    for 迭代变量 in 可迭代数集
        #语句块
    end
格式3:
    for 迭代变量∈可迭代数集
        #语句块
    end
```

注意:

- 三个遍历操作符=、in及∈是等价的，可以任选一个；
- ∈输入：\in + tab键
- 可迭代的内容包括元组、字典、集合、数组、及列表等



3.3 Julia脚本构建-控制流

多个迭代变量:

```
julia> for i = 1:2, j in (1, 2, 3)
    print((i, j))
end
(1, 1)(1, 2)(1, 3)(2, 1)(2, 2)(2, 3)
```

```
julia> for i in [1 2 3], j = 3:4
    print((i, j))
end
(1, 3)(1, 4)(2, 3)(2, 4)(3, 3)(3, 4)
```

```
julia> for j = 3:4, i in [1 2 3]
    print((i, j))
end
(1, 3)(2, 3)(3, 3)(1, 4)(2, 4)(3, 4)
```

等价

```
julia> for i = 1:2
    for j in (1, 2, 3)
        print((i, j))
    end
end
(1, 1)(1, 2)(1, 3)(2, 1)(2, 2)(2, 3)
```

```
julia> for i=1:2;j=3:4
    print((i,j))
end
(1, 3:4)(2, 3:4)
```

说明:

- for循环支持同时对多个数据集进行遍历;
- 遍历操作符的顺序影响内部语句循环变量的取值。

注意:

- 每一个遍历表达式之间的分割符是逗号而非分号, 当为分号时, 这种表达式将为复合表达式而非遍历表达式。



3.3 Julia脚本构建-控制流

注意:

在 for 循环内，它就只在 for 循环内部可见，在外部和后面均不可见。你需要一个新的交互式会话实例或者一个新的变量名来测试这个特性：

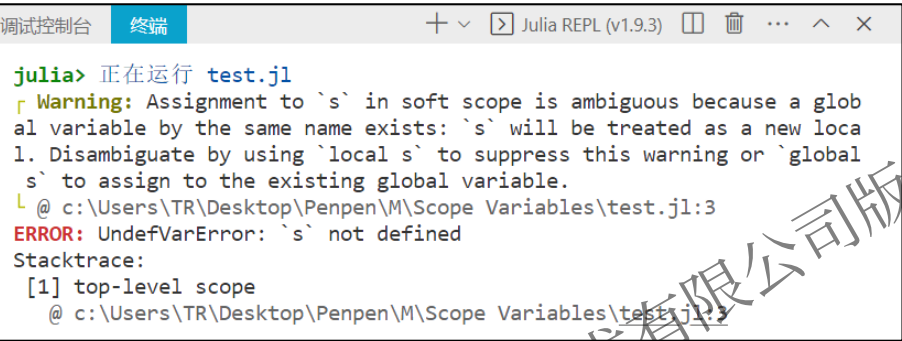
```
s = 0 # global `s`
for i = 1:5
    s=s+i # new local `s`
end
```

解决方法一：使用 local 和 global 标识符
能够帮助 Julia 准确判断用户想使用的变量所在的作用域、提高代码的可读性。

```
s = 0 # global `s`,
for i = 1:5
    global s
    s=s+i # new local `s`
end
```

解决方法二：封装成函数
脚本文件封成函数能够隔绝全局作用域，避免全局作用域的和软局部作用域里的变量相互影响。

```
function foo()
    s = 0 # local `s`
    for i = 1:5
        s = s + i # assign existing local `s`
    end
end
foo()
```



3.3 Julia脚本构建-控制流

break—强制退出循环

```
julia> i=1
1

julia> while i < 10
    if i > 5
        break
    end
    print(i)
    i += 1
end
12345
```

说明：

- break强制循环在满足控制条件时退出。
- break在for循环中的使用与while循环相同。

continue—强制跳过当前迭代

```
julia> i=1
1

julia> while i < 10
    i += 1
    if i < 5
        continue
    end
    print(i)
end
5678910

julia> i
10
```

说明：

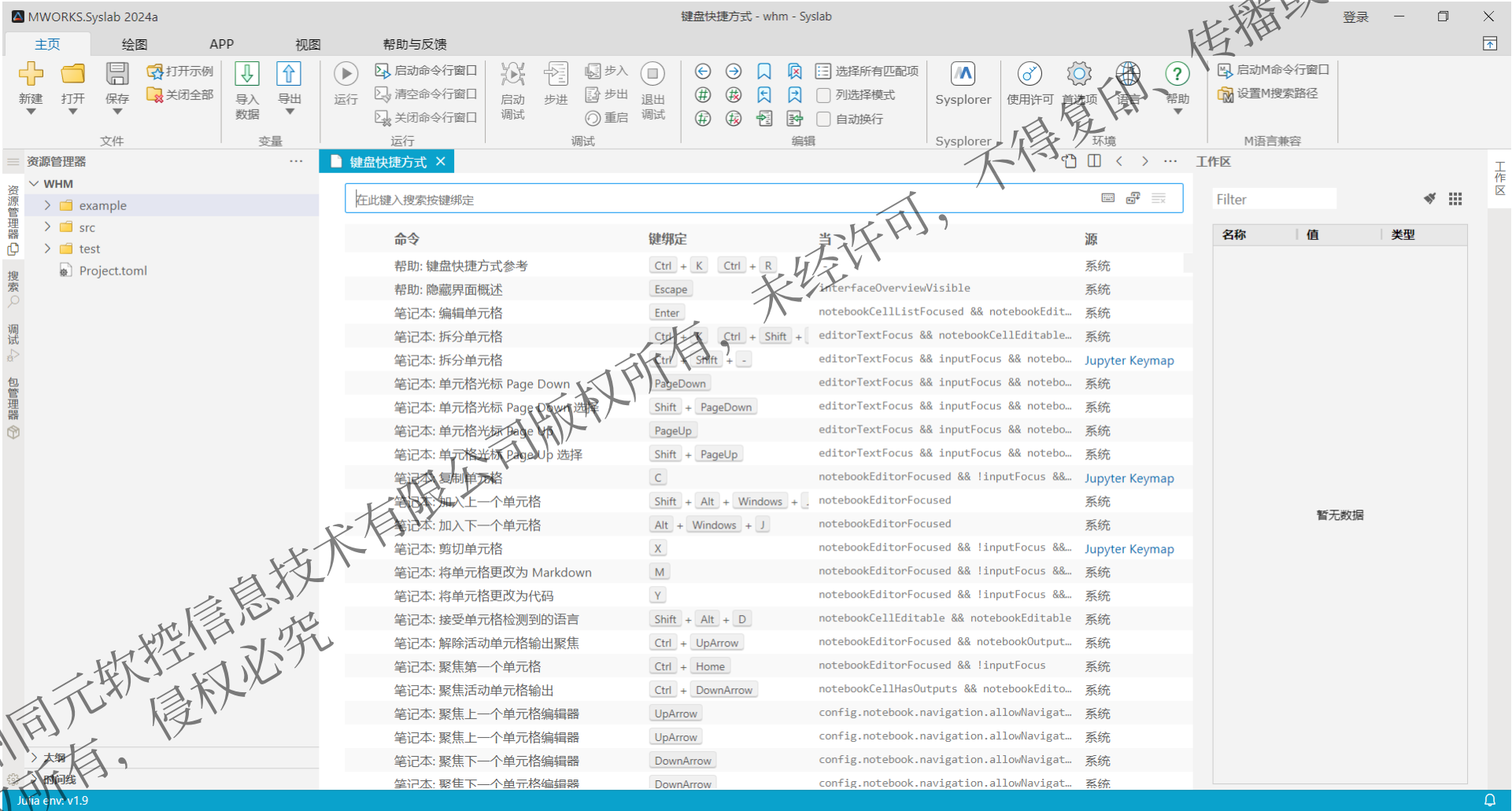
- continue可以让循环跳过当前迭代，直接进入下一个循环的迭代。
- continue在for循环中的使用与while循环相同。



3.4 键盘快捷方式

快捷键设置

点击左侧边栏下方的 [设置>键盘快捷方式]，打开 键盘快捷方式 页面，可以浏览和编辑本平台的所有快捷键。



Part 4

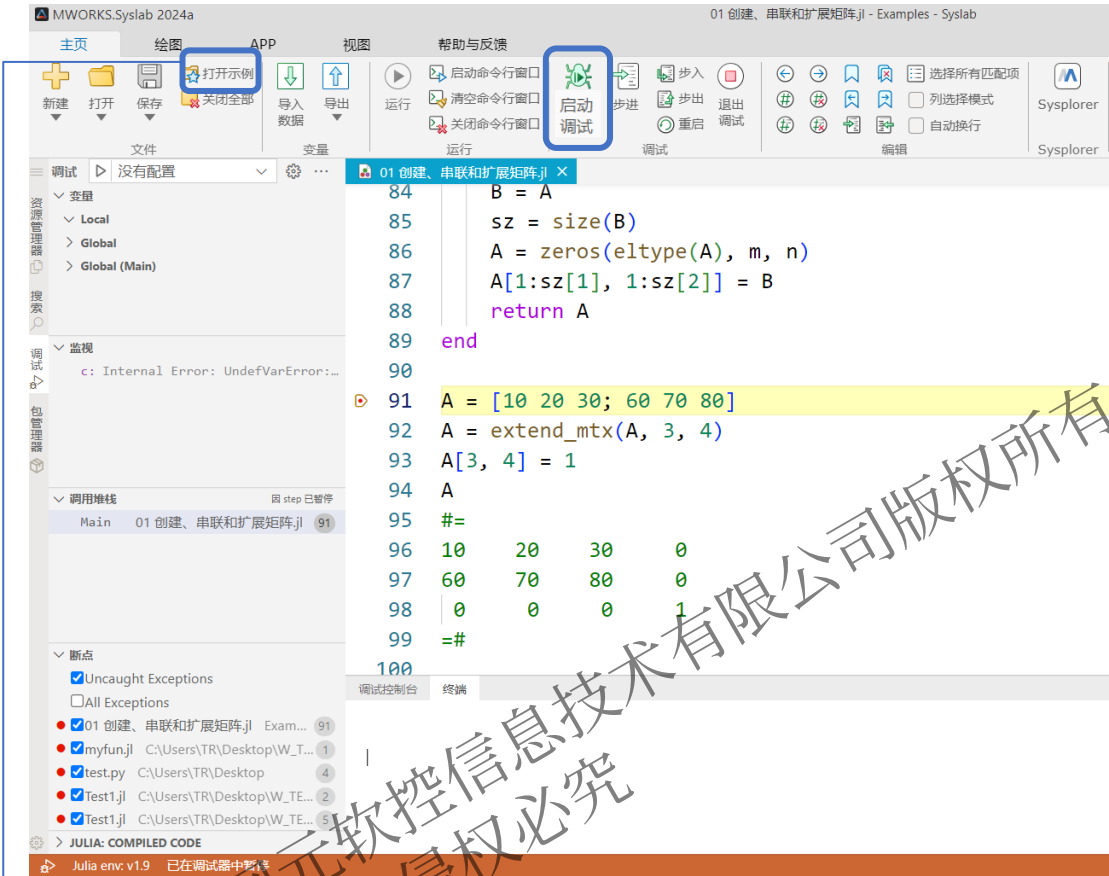
脚本调试

1. 代码调试器
2. 步进
3. 步入
4. 调试窗口
5. 调试控制条

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

4.1 代码调试器

Syslab提供代码调试器，支持对代码文件的单步调试、断点调试、添加监视、查看调用堆栈等，并提供交互式调试控制台。



操作步骤：

- 在脚本打上断点
- 点击菜单栏中“启动调试”或**快捷键F5**进入DeBug模式

说明：

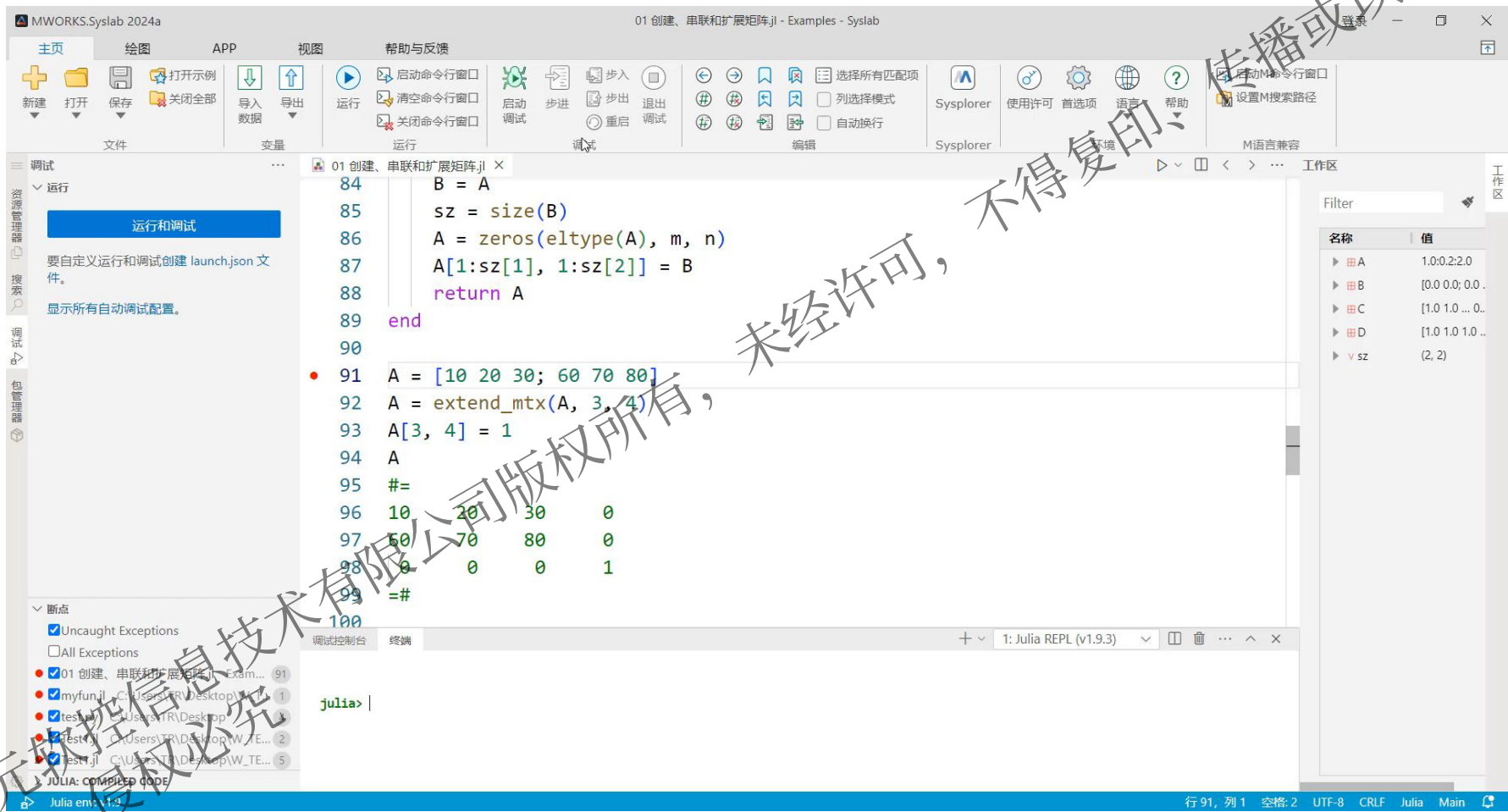
- 继续(F5)：启动调试或者继续运行调试。
- 步进(F10)：单步执行遇到子函数时不会进入子函数内，而是将子函数整个执行完再停止。
- 步入(F11)：单步执行遇到子函数就进入并且继续单步执行。
- 步出(Shift+F11)：当单步执行到子函数内时，执行完子函数余下部分，并返回到上一层函数。
- 重启(Ctrl+Shift+F5)：重新启动调试。
- 退出调试(Shift+F5)：停止调试。

示例见：Syslab内置：主页/打开示例/EXAMPLES/02语言基础知识/02矩阵和数组/01创建、串联和扩展矩阵



4.2 步进

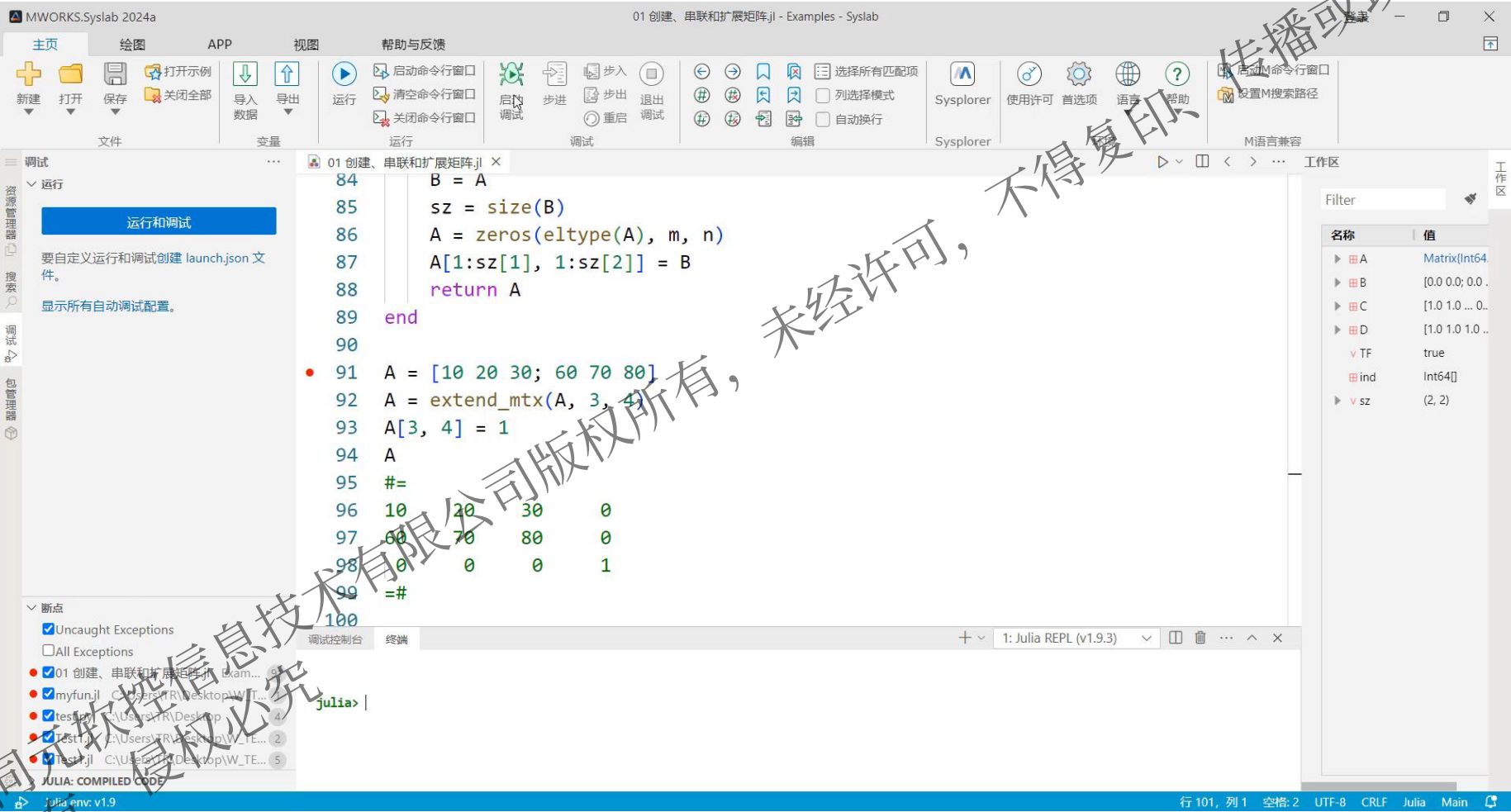
步进(F10)：单步执行遇到子函数时不会进入子函数内，而是将子函数整个执行完再停止。



步进演示.mp4

4.3 步入

步入(F11): 单步执行遇到子函数就进入并且继续单步执行。



步入演示.mp4

4.4 调试窗口

说明:

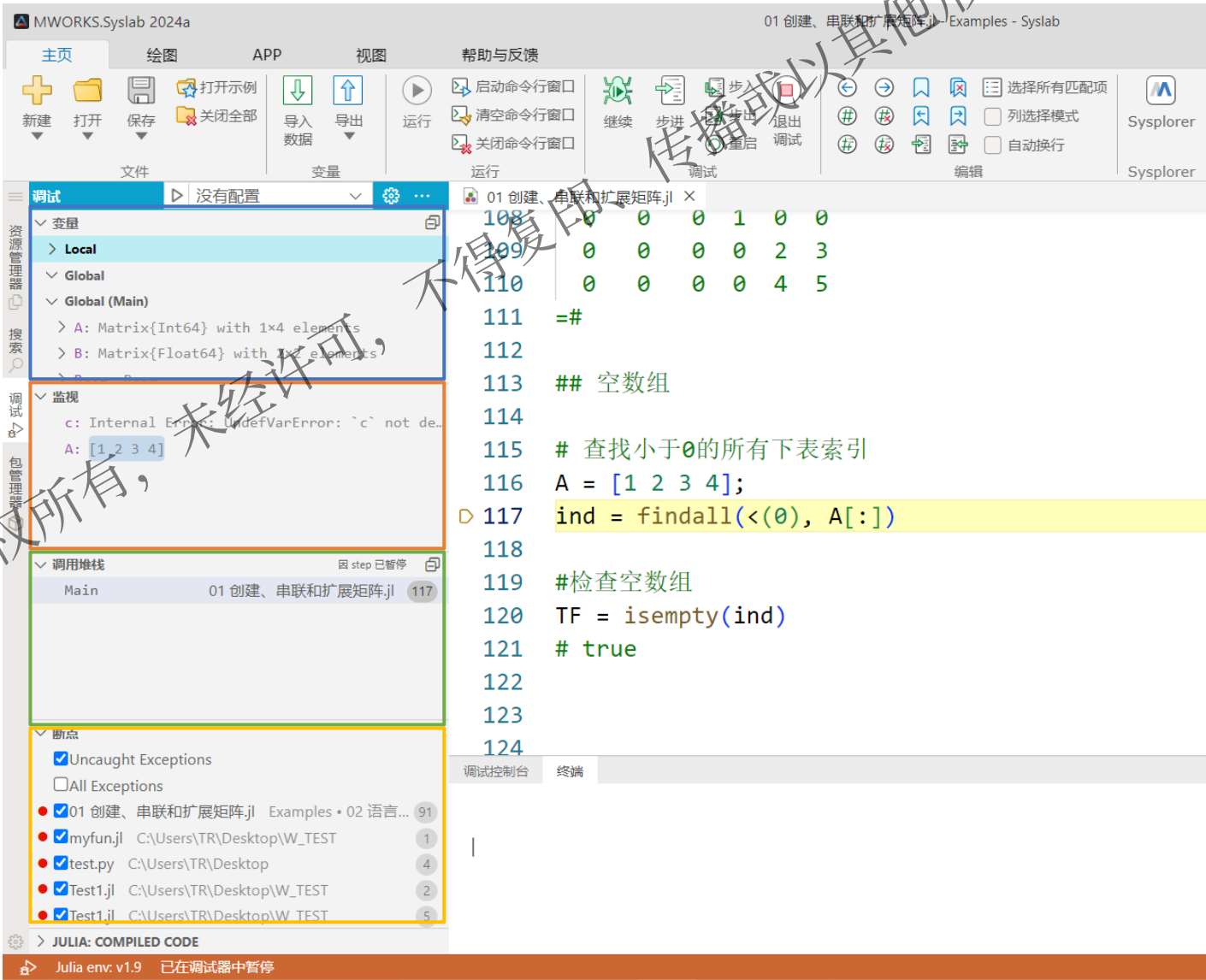
变量:

- Local: 函数局部变量的值
- Global: 函数全局变量的值, 变量前具有 global 前缀
- Global(Main): 主程序全局变量的值

监视: 输入变量名, 监视变量的变化

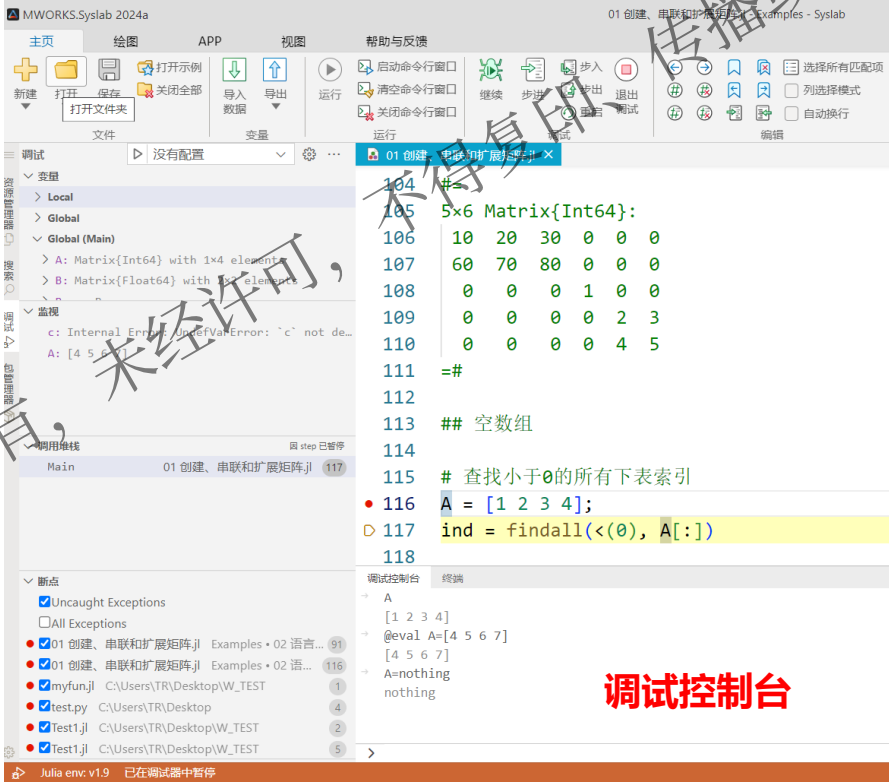
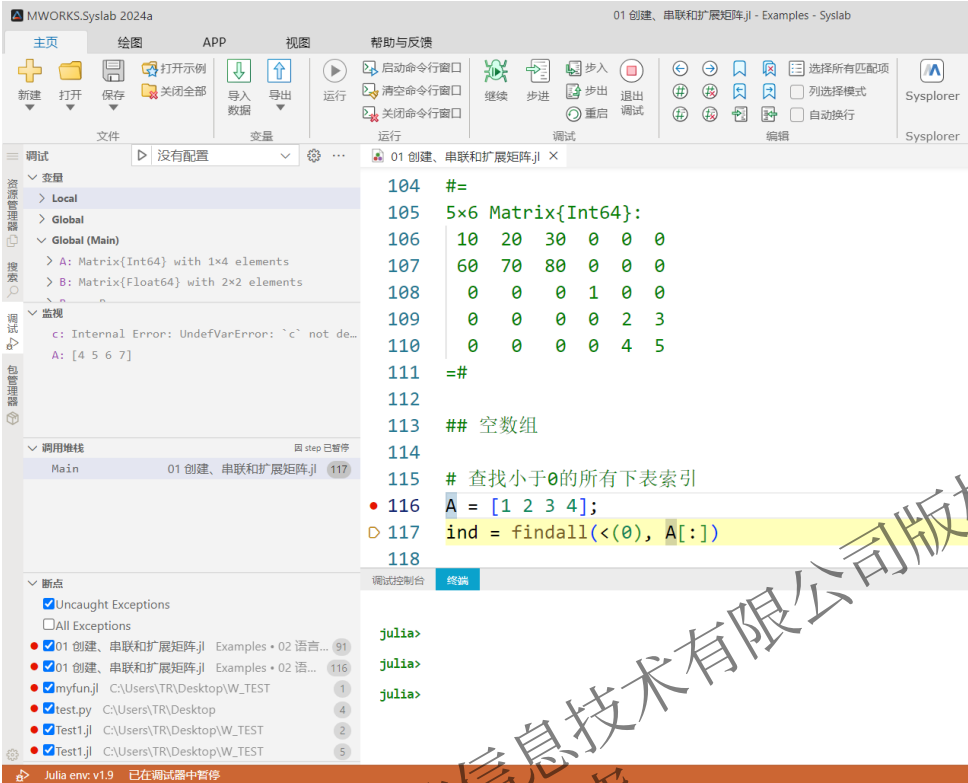
调用堆栈: 用于了解函数间的调用关系和逻辑

断点: 模型中所有断点的信息



4.5 调试控制台

在调试过程中，通过调试控制台能够对变量进行增/删/改/查



- 查看变量值：输入变量名；
- 修改全局变量值：@eval(变量名=变量值)
- 修改局部变量值：@eval \$(变量名=变量值)
- 释放变量：变量名=nothing

Part 5

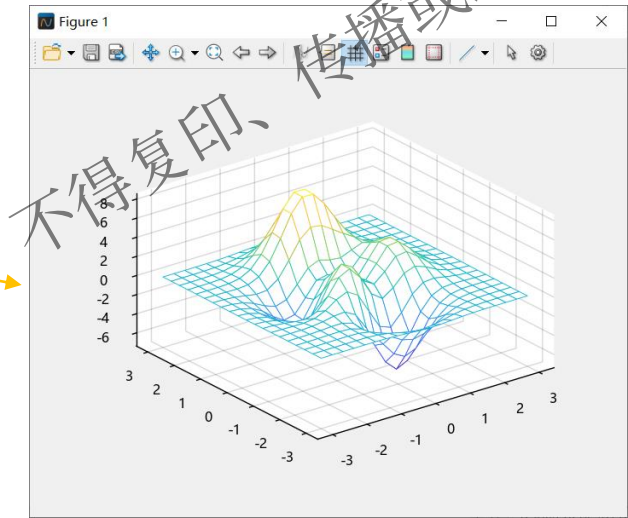
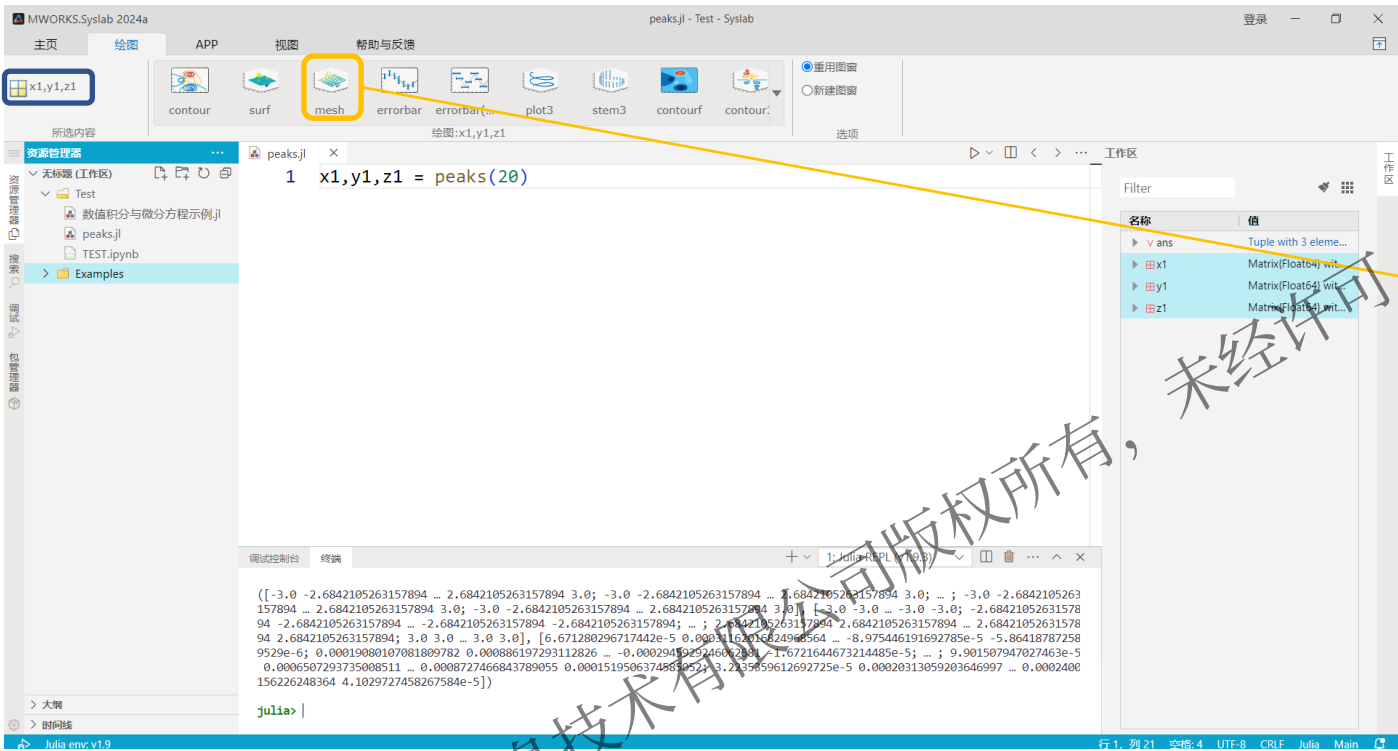
结果后处理

1. 数据可视化
2. 结果打印与保存
3. 数据导入与导出

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

5.1 结果后处理-数据可视化(图形化操作)

Syslab绘图页中具备多种类型图形绘制功能，可直接选择变量进行绘制



选择工作区变量，可视化绘图步骤：

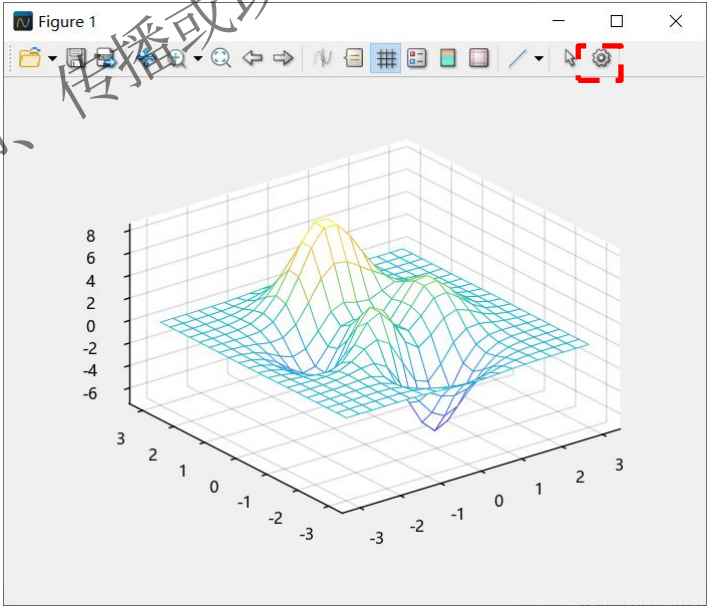
1. 工作空间左键选择绘图的变量
2. 在“绘图”菜单栏选择图类型

`x1,y1,z1 = peaks(20)` #分别返回符合高斯分布的20*20的矩阵

5.1 结果后处理-数据可视化(图形化操作)

标签与注释: 添加标题、轴标签、信息文本及图表注释

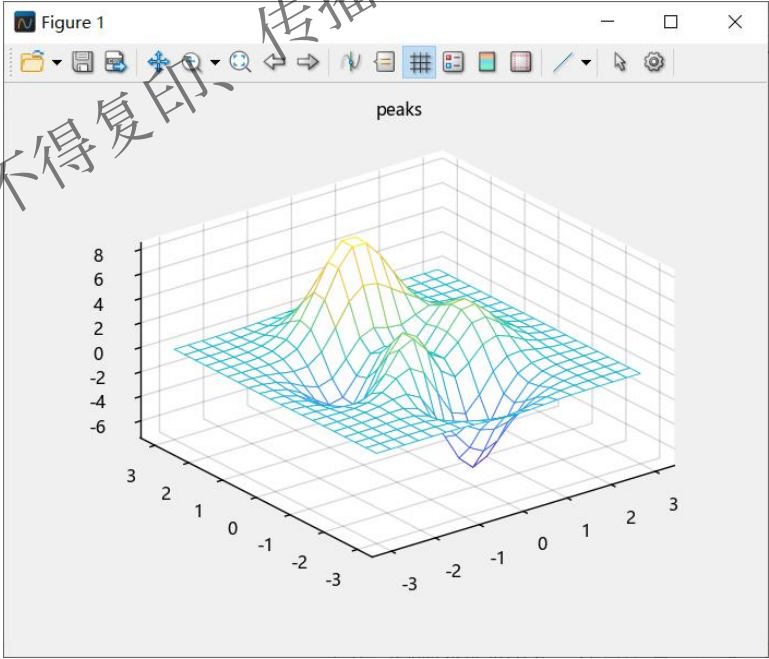
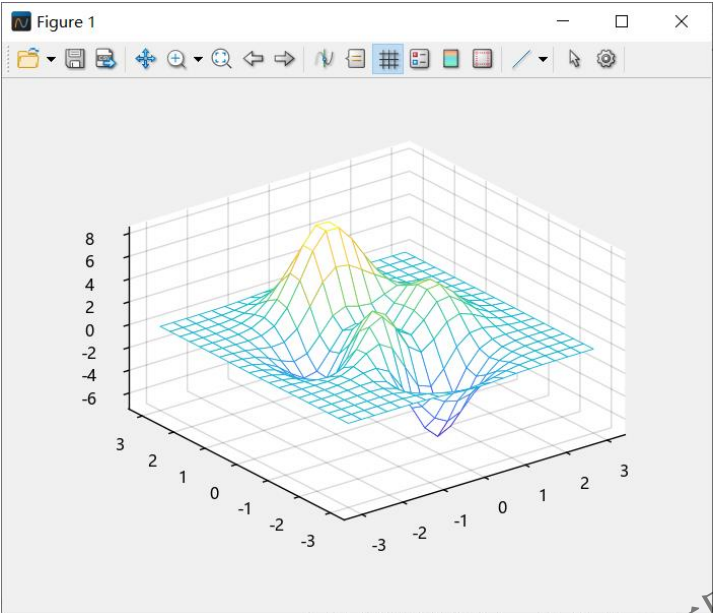
类型	Julia函数	功能	可视化操作
标签	title	添加标题	属性检查器->Axes->标签->Title->Text
	sgtitle	在子图网格上添加标题	\
	xlabel	为x轴添加标签	属性检查器->Axes->标签->XLabel->Text
	ylabel	为y轴添加标签	属性检查器->Axes->标签->YLabel->Text
	zlabel	为z轴添加标签	属性检查器->Axes->标签->ZLabel->Text
	legend	在坐标区上添加图例	属性检查器->Axes->标签->Legend->Text
注释	text	向数据点添加文本说明	
	gtext	使用鼠标将文本添加到图窗	
	xline	具有常量 x 值的垂直线	\
	yline	具有常量 y 值的水平线	     
	annotation	创建注释	
	datatip	创建数据提示	直接点击图窗数据
	line	创建基本线条	
	rectangle	创建带有尖角或圆角的矩形	
	ginput	标识坐标区坐标	\



说明: 图片背景色、标记、字体颜色线型等均可通过图窗进行设置。

5.1 结果后处理-数据可视化(图形化操作)

可在图形窗口直接配置属性，如增加标题、坐标轴标签等。

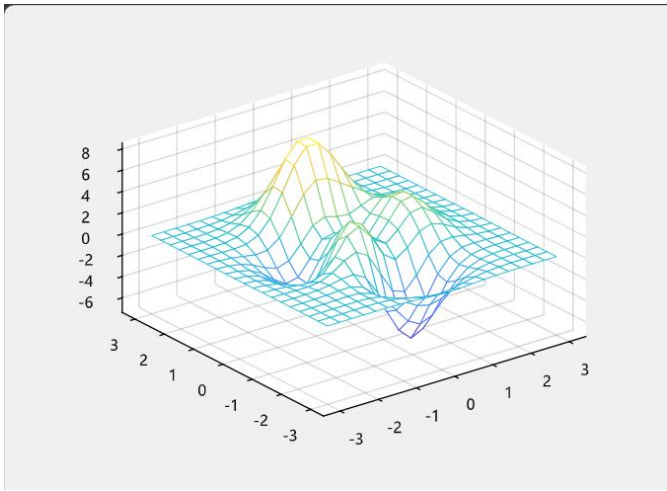
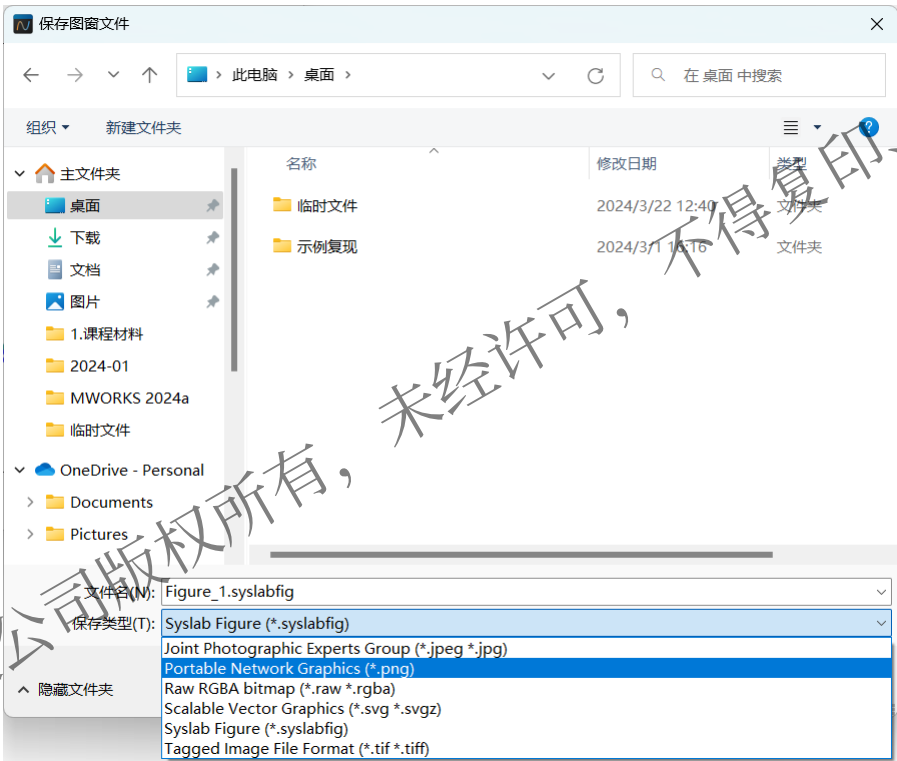
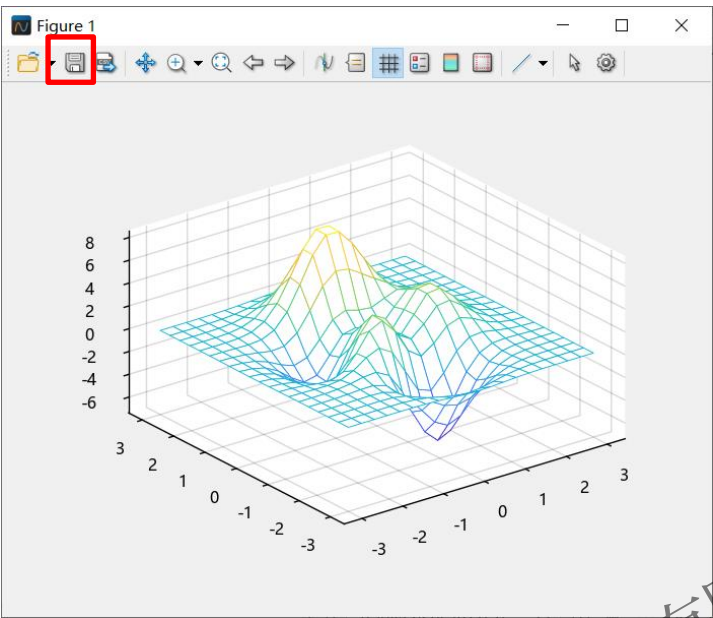


步骤:

1. 添加标题
2. 添加X、Y、Z轴标签

5.1 结果后处理-图片保存(图形化操作)

在图形窗口中可以将图片以制定格式保存至本地。



步骤:

- 1. 点击图片保存
- 2. 设置存储位置、文件名及保存类型
- 3. 查看保存图片



5.1 结果后处理-数据可视化(脚本)

二维图分类: 线图、对数图、函数图、直方图、散点图、饼图和热图、条形图、针状图、极坐标图和等高线图。

图形类型	函数	说明
线图	plot	二维线图
	stairs	阶梯图
	errorbar	含误差条的线图
	area	填充区二维绘图
对数图	loglog	双对数刻度图
	semilogx	半对数图(x轴有对数刻度)
	semilogy	半对数图(y轴有对数刻度)
函数图	fplot	绘制表达式或函数
	fimplicit	绘制隐函数
直方图	histogram	直方图
	boxchart	创建箱线图

图形类型	函数	说明
散点图	scatter	散点图
	spy	可视化矩阵的稀疏模式
	plotmatrix	散点图矩阵
饼图和热图	pie	饼图
	heatmap	创建热图
	sortx	对热图行中的元素进行排序
	sorty	对热图列中的元素进行排序
	wordcloud	使用文本数据创建文字云图
条形图	bar	条形图
	barh	水平条形图
	pareto	帕累托图
针状图	stem	针状图

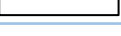
图形类型	函数	说明
极坐标图	polarplot	在极坐标中绘制线条
	polarscatter	极坐标中的散点图
	polarhistogram	极坐标中的直方图
	ezpolar	易用的极坐标绘图函数
等高线图	contour	矩阵的等高线图
	contourf	填充的二维等高线图
	fcontour	绘制等高线



5.1 结果后处理-数据可视化(脚本)

可以直接使用plot函数直接绘制二维线图，一般形式为plot(x1,y1,"s1", x2,y2,"s2"),s1、s2为曲线的特征

线型	说明	表示的线条
"-"	实线	————
"--"	虚线	-----
":"	点线	
"-."	点划线	- . - . - .

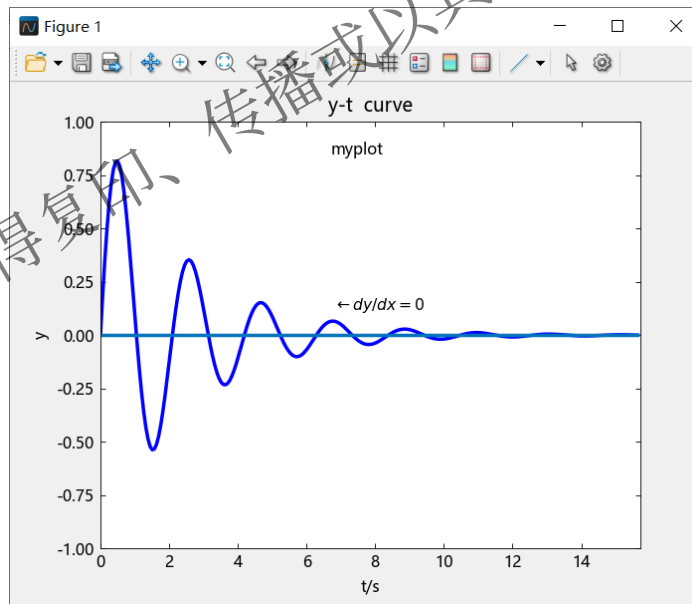
颜色名称	短名称	颜色
red	r	
green	g	
blue	b	
cyan	c	
magenta	m	
yellow	y	
black	k	
white	w	

标记	符号	说明
"."	●	空心小圆
"o"	○	空心大圆
"v"	▼	下三角
"^"	▲	上三角
"<"	◀	左三角
">"	▶	右三角
"*"	★	星号
"x"	×	叉号
"s"	■	正方形
"d"	◆	菱形

5.1 结果后处理-数据可视化(脚本)

绘制衰减图线 $y=e^{-t/2.5}\sin 3t$,其中 t 的取值范围为 $[0,5\pi]$

```
#定义自变量t的取值范围
t = 0:pi/50:5*pi
#计算对应于自变量数组的y数组, 使用广播
y = exp.(-t / 2.5) .* sin.(3t)
#绘制特征为实线、蓝色、线宽为2的曲线和0参考线
plot(t, y, "-b", linewidth = 2,t,zeros(size(t)))
#标注输出图线的最大值最小值,x轴为[0,5*pi], y轴为[-1,1]
axis([0, 5 * pi, -1, 1])
#定义xy坐标轴的名称
xlabel("t/s")
ylabel("y")
#定义图形名称
title("y-t curve")
#在(6.77,0.12)位置处插入"←dy/dx=0"
text(6.77, 0.12, raw"$\leftarrow dy/dx=0$")
#鼠标选择的位置插入文本"myplot"
gtext("myplot")
```



title —— 给图形加标题
xlabel —— 给x轴加标注
ylabel —— 给y轴加标注
text —— 在图形指定位置加标注
gtext —— 将标注加到鼠标点击位置
grid ("on/off") —— 打开、关闭坐标网格线
legend —— 添加图例
axis —— 控制坐标轴的刻度

5.1 结果后处理-数据可视化(脚本)

三维图分类: 线图、函数图、散点图、条形图、针状图、等高线图、曲面图和网格图等。

类型	函数	说明
线图	plot3	三维点或线图
函数图	fplot3	三维参数化曲线绘图函数
散点图	scatter3	三维散点图
条形图	bar3	绘制三维条形图
	bar3h	绘制水平三维条形图
针状图	stem3	三维针状图
等高线图	contour3	三维等高线图
曲面图和网格图	surf	曲面图
	mesh	网格曲面图
	peaks	包含两个变量的示例函数
	fsurf	绘制三维曲面
	fmesh	绘制三维网格图

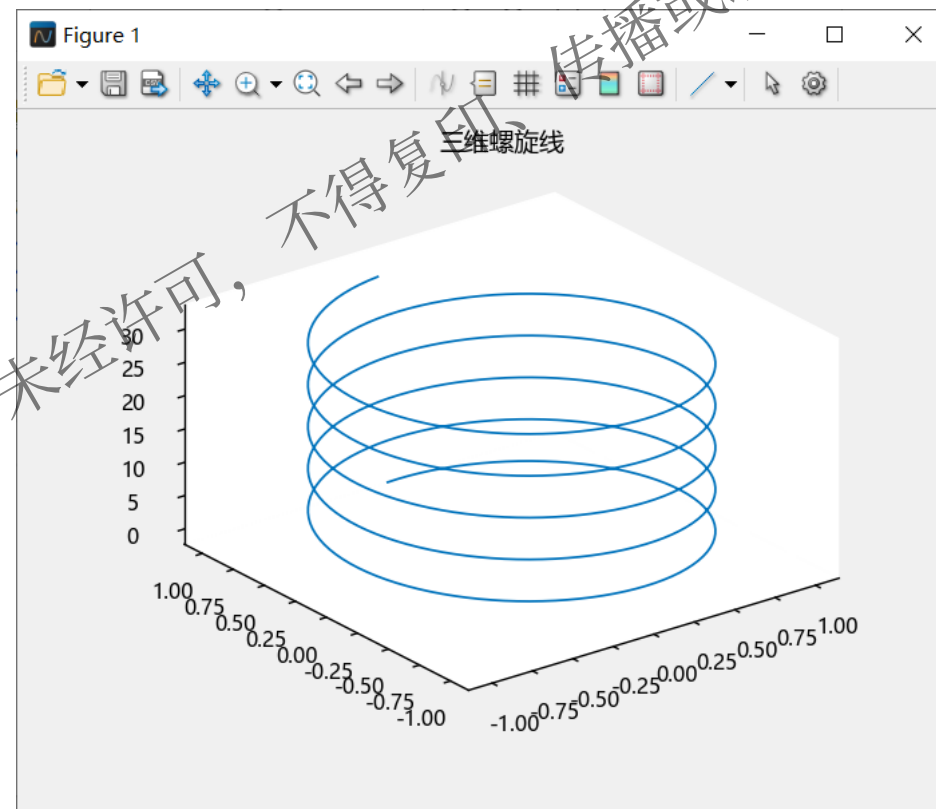


5.1 结果后处理-数据可视化(脚本)

可以直接使用plot3函数直接绘制三维线图，一般形式为`plot3(x1,y1,z1,"s")`, s为曲线的特征

绘制三维螺旋线

```
#定义自变量t的取值范围
t = 0:pi/50:10*pi;
#定义xy坐标与t的关系
x = sin.(t);
y = cos.(t);
plot3(x, y, t);
title("三维螺旋线")
```

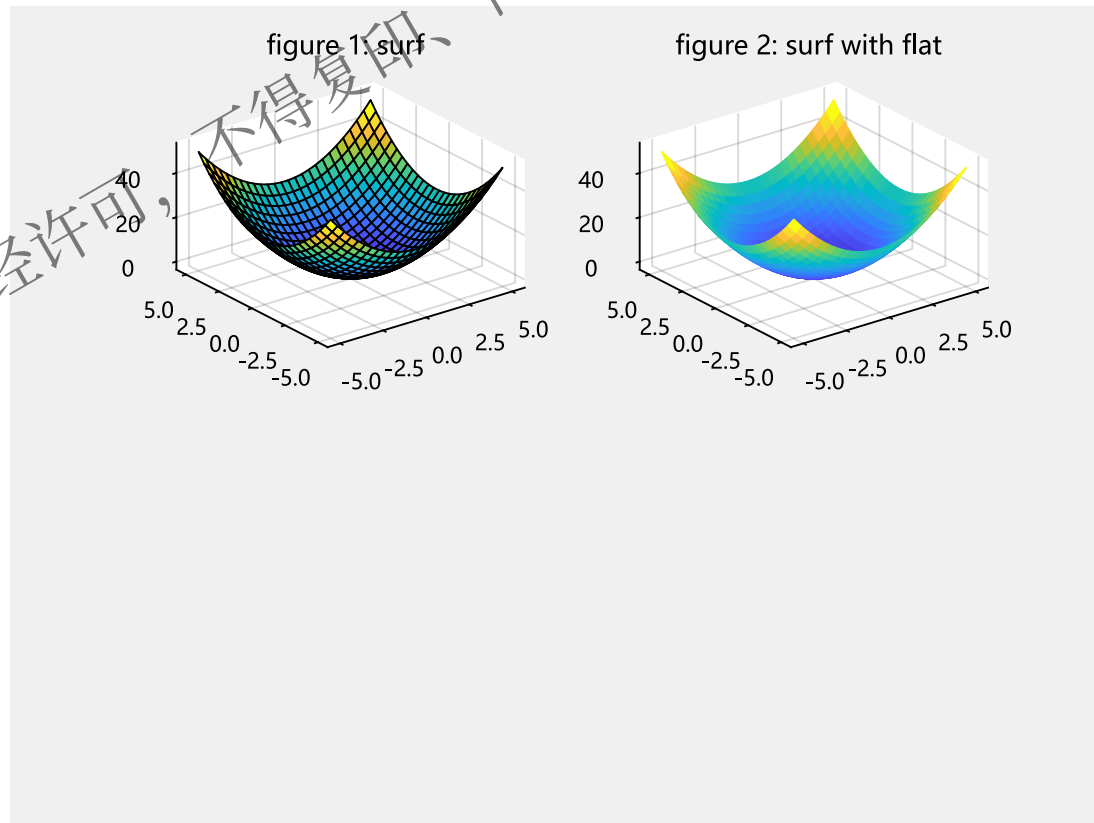


5.1 结果后处理-数据可视化(脚本)

可以使用subplot将图形区进行分区，可在各个分区中分别进行绘图

在一个图形区分别绘制普通曲面图，带平面阴影的曲面图，无网格边界的曲面图和部分被遮挡的曲面图

```
x = -5:0.2:5;  
y = x;#定义向量x和y  
X, Y = meshgrid(x, y);#生成坐标  
Z = X.^ 2 + Y.^ 2;#表达式点运算  
#图1: 普通绘制  
#subplot将多个图画到同一平面  
subplot(2, 2, 1);  
s = surf(X, Y, Z);#绘制三维图形  
title("figure 1: surf");  
#图2: 平面阴影  
subplot(2, 2, 2);  
s = surf(X, Y, Z);  
#set_edgcolor()用来修改网格属性, flat修饰网格  
s.set_edgcolor("flat")  
title("figure 2: surf with flat");
```



5.1 结果后处理-数据可视化(脚本)

可以使用subplot将图形区进行分区，可在各个分区中分别进行绘图

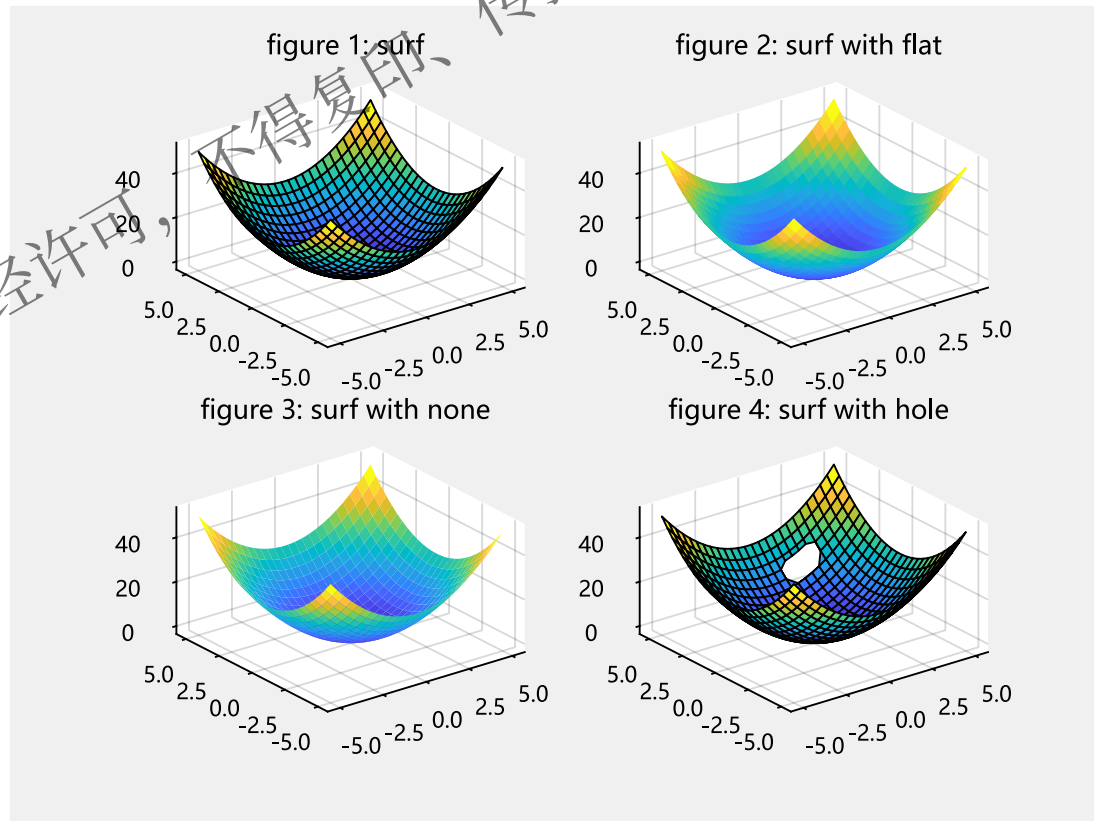
在一个图形区分别绘制普通曲面图，带平面阴影的曲面图，无网格边界的曲面图和部分被遮挡的曲面图

#图3: 无网格边界

```
subplot(2, 2, 3);  
s = surf(X, Y, Z);  
#none表示无网格边界  
s.set_edgecolor("none")  
title("figure 3: surf with none");
```

#图4: 遮挡绘制

```
subplot(2, 2, 4);  
x1 = X[1, :];  
y1 = Y[:, 1];  
i = find((y1 .> 3) .& (y1 .< 4));  
j = find((x1 .> 1) .& (x1 .< 3));  
#赋空值  
Z[i, j] .= NaN;  
s = surf(X, Y, Z);  
title("figure 4: surf with hole");
```



5.1 结果后处理-打印保存

函数总览

函数	说明
plt_print	打印图窗或保存为特定文件格式
saveas	将图窗保存为特定文件格式

saveas:

语法	说明
saveas(fig,filename)	将fig指定的图窗保存到filename文件中。

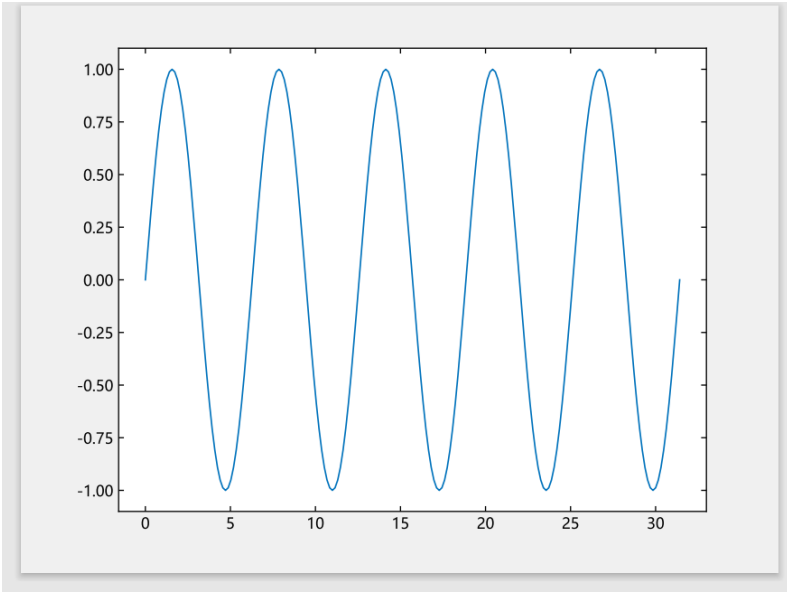
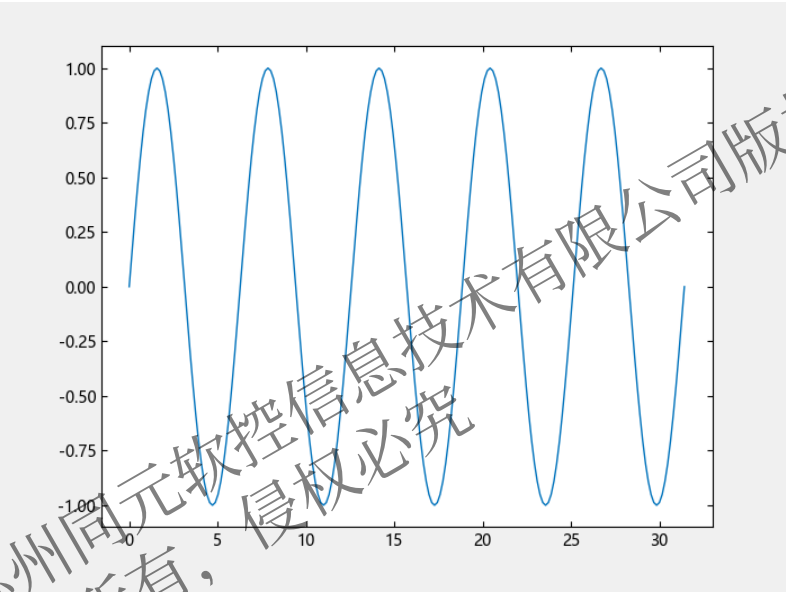
参数说明:

函数	说明
fig	要保存的图窗，指定为图窗对象。
filename	文件名，指定为字符向量或字符串，包括或不包括文件扩展名均可。

5.1 结果后处理-打印保存

```
t = 0:pi/20:10*pi;  
y = sin.(t);  
plot(t,y);  
#将图片以pdf格式保存  
saveas(gcf(), "ty1.pdf");
```

06 SyslabWorkspace	2023/5/28 17:19	文件夹
07 Interfaces	2023/5/28 17:19	文件夹
10 AppDemos	2023/5/28 17:19	文件夹
ty1.pdf	2023/5/29 0:58	Microsoft Edge ... 8 KB

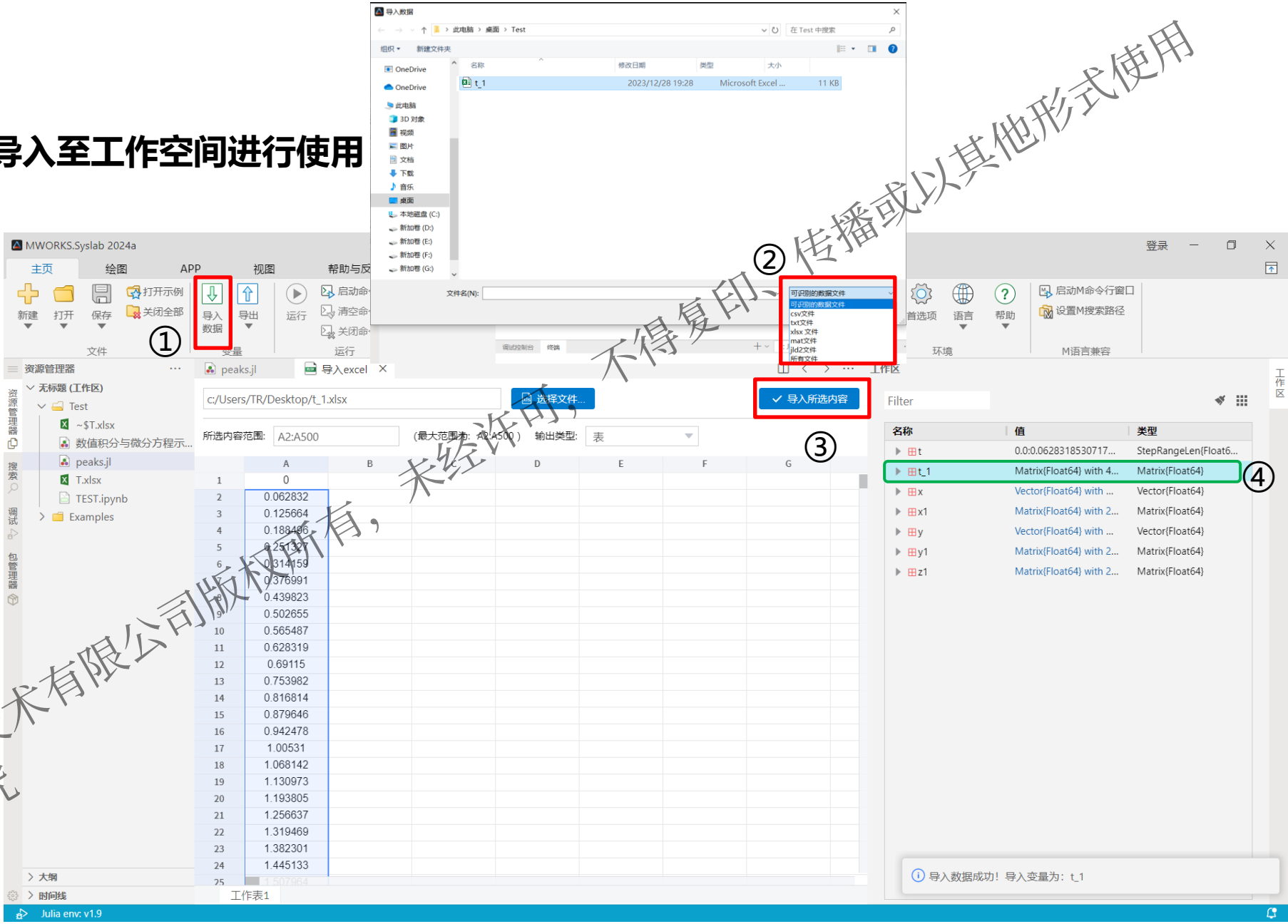


5.2 结果导入

数据导入：可将数据文件直接导入至工作空间进行使用

操作步骤：

- 点击“导入”，选择文件格式：
 - 文本：txt、csv
 - Excel：xlsx
 - Mat
 - jld2
- 选择文件并导入数据
- 工作空间查看结果，并可对结果重命名



5.3 结果导出

数据导出：所有计算结果和部分变量结果可直接导出为csv格式

操作步骤：

- 方式1：导出所有结果
- 方式2：导出单个结果



@苏州同元软控信息技术有限公司
版权所有，侵权必究



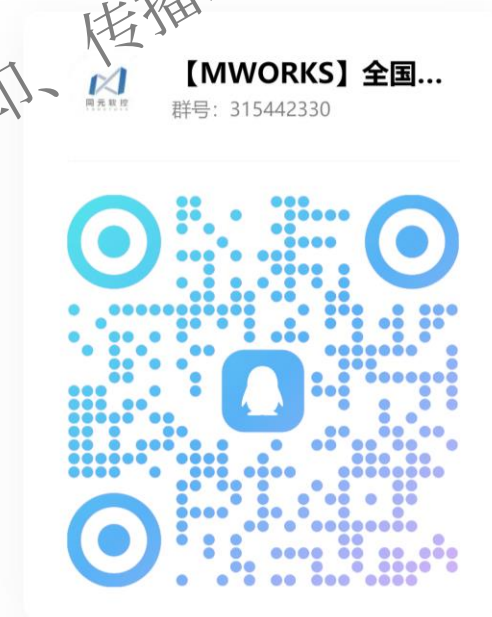
建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入中国创新，打造先进软件

感谢聆听

同元软控官方培训视频：点击链接
<https://www.tongyuan.cc/education>即可查看



【MWORKS】全国大学生数学建模竞赛
技术交流群



推荐课程：

1. MWORKS.Syslab基础课程：《软件基础功能(Syslab)》、《Julia语法》
2. 数学与基础应用专项课程：《基础数学工具箱应用》、《符号数学工具箱应用》、《图形工具箱应用》、《曲线拟合工具箱应用》、《统计工具箱应用》